



Documentação Técnica - TechChallenge Fase 3



Repositório 1: TechChallenge-infra-foundational

Objetivo

Este é o **primeiro repositório** a ser executado e representa a **fundação de toda a arquitetura**. Ele cria a "fortaleza" da aplicação: uma rede virtual isolada e segura na Azure, onde todos os outros componentes serão implantados.



O que este repositório faz?

Analogia

Imagine que você vai construir um condomínio fechado de luxo:

- **VNet (Rede Virtual)** = O terreno murado do condomínio
 - **Subnets (Sub-redes)** = Lotes divididos dentro do terreno para diferentes finalidades
 - **NSG (Network Security Group)** = Portaria com regras de quem pode entrar e sair
 - **Storage Account para Terraform State** = O arquivo da planta do condomínio guardado em lugar seguro
-



Estrutura de Arquivos

main.tf - Configuração Principal

1. Provider e Versões

```
hcl

terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}
```

- Define que vamos usar o **provider Azure** (azurerm)
 - Versão ~> 3.0 significa "qualquer versão 3.x"
-

2. Resource Group Principal

```
hcl

resource "azurerm_resource_group" "app_rg" {
  name     = "rg-tchungry-prod"
  location = "Brazil South"
}
```

O que é: Container lógico que agrupa todos os recursos da aplicação.

Por que: Facilita gerenciamento, billing e organização. Podemos deletar tudo de uma vez se necessário.

Nomenclatura: `rg-tchungry-prod`

- `rg` = Resource Group
 - `tchungry` = Nome do projeto
 - `prod` = Ambiente de produção
-

3. Rede Virtual (VNet)

```
hcl

resource "azurerm_virtual_network" "vnet" {
  name                = "vnet-tchungry-prod"
  resource_group_name = azurerm_resource_group.app_rg.name
  location            = azurerm_resource_group.app_rg.location
  address_space       = ["10.0.0.0/16"]
}
```

O que é: Uma rede privada isolada na Azure.

Por que:

- **Segurança:** Nenhum recurso fica exposto diretamente na internet
- **Isolamento:** Nossa aplicação fica numa rede privada, separada de outras redes
- **Controle:** Decidimos quem pode se comunicar com quem

CIDR 10.0.0.0/16:

- Permite até **65.536 endereços IP** (de 10.0.0.0 até 10.0.255.255)
 - Espaço suficiente para crescimento futuro
-

4. Subnet para AKS (Kubernetes)

```
hcl

resource "azurerm_subnet" "aks_subnet" {
  name                = "snet-aks"
  resource_group_name = azurerm_resource_group.app_rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes    = ["10.0.1.0/24"]
}
```

O que é: "Lote" dedicado para o cluster Kubernetes.

Por que:

- **Isolamento:** O Kubernetes fica numa área separada
- **Segurança:** Podemos aplicar regras específicas só para essa subnet
- **Organização:** Fácil identificar quais IPs pertencem ao K8s

CIDR 10.0.1.0/24:

- Permite até **256 IPs** (10.0.1.0 a 10.0.1.255)
 - Suficiente para os pods e serviços do Kubernetes
-

5. Subnet para Private Endpoints

```
hcl

resource "azurerm_subnet" "private_endpoints_subnet" {
  name                = "snet-private-endpoints"
  address_prefixes    = ["10.0.4.0/24"]
}
```

O que é: Subnet dedicada para conexões privadas com serviços Azure PaaS.

Por que:

- **Segurança máxima:** SQL Database e Storage Account podem ser acessados via IP privado (não pela internet)
- **Compliance:** Dados sensíveis nunca trafegam pela internet pública
- **Performance:** Conexão direta via backbone da Azure

Exemplo: O AKS pode acessar o SQL Database pelo IP 10.0.4.X em vez de `xxx.database.windows.net` público.

6. Subnet para Aplicações (App Gateway, Functions)

```
hcl

resource "azurerm_subnet" "app_subnet" {
  name            = "snet-apps"
  address_prefixes = ["10.0.2.0/24"]
}
```

O que é: Subnet para recursos de aplicação que precisam estar na VNet mas não são Kubernetes.

Por que:

- **Azure Functions:** Integração VNet para acesso privado aos backends
- **Application Gateway:** Ingress Controller precisa de subnet dedicada
- **Flexibilidade:** Espaço para outros serviços futuros

7. Subnet Exclusiva para API Management (APIM)

```
hcl

resource "azurerm_subnet" "apim_subnet" {
  name            = "snet-apim"
  address_prefixes = ["10.0.3.0/24"]
}
```

O que é: Subnet dedicada EXCLUSIVAMENTE para o API Management.

Por que separar o APIM?

1. **Requisito da Azure:** APIM em VNet exige subnet dedicada (não pode compartilhar)
2. **Segurança:** Podemos aplicar NSG específico com regras rigorosas
3. **Escalabilidade:** APIM pode crescer sem afetar outras subnets
4. **Troubleshooting:** Facilita debug de problemas de rede

8. Network Security Group (NSG) para APIM

```
hcl
```

```
resource "azurerm_network_security_group" "apim_subnet_nsg" {  
  name = "nsg-snet-apim"  
  
  # Regra 1: Management do APIM (porta 3443)  
  security_rule {  
    name           = "AllowApiManagement"  
    priority       = 100  
    direction      = "Inbound"  
    access         = "Allow"  
    protocol        = "Tcp"  
    destination_port_range = "3443"  
    source_address_prefix = "ApiManagement"  
    destination_address_prefix = "VirtualNetwork"  
  }  
}
```

O que é: Firewall de camada 4 (rede) que controla tráfego de entrada/saída.

Regras Críticas Explicadas:

Regra 1: Management do APIM (porta 3443)

- **Por que:** Azure precisa gerenciar o APIM (updates, health checks)
- **Source:** `ApiManagement` = service tag da Azure para o plano de controle
- **Priority 100:** Alta prioridade (números baixos = avaliadas primeiro)

Regra 2: HTTPS da Internet (porta 443)

```
hcl  
  
security_rule {  
  name           = "AllowHTTPSInbound"  
  priority       = 110  
  destination_port_range = "443"  
  source_address_prefix = "Internet"  
}
```

- **CRÍTICO!** Sem essa regra, ninguém consegue acessar a API
- Permite que clientes externos (aplicação web, Postman, etc.) façam requests HTTPS

Regra 3: Azure Load Balancer

```
hcl
```

```
security_rule {
  name           = "AllowAzureLoadBalancer"
  priority       = 120
  source_address_prefix = "AzureLoadBalancer"
}
```

- **Por que:** APIM usa Load Balancer interno para distribuir tráfego
- Sem isso, o APIM fica "unhealthy"

Regra 4: Outbound - APIM para Backends

```
hcl

security_rule {
  name           = "AllowAPIMToBackends"
  direction       = "Outbound"
  destination_port_ranges = ["80", "443", "1433", "3306"]
}
```

- **Portas:**
 - **80/443**: HTTP/HTTPS para chamar APIs e Functions
 - **1433**: SQL Server (se APIM precisar consultar DB diretamente)
 - **3306**: MySQL (preparado para futuros serviços)

9. Associação NSG ↔ Subnet

```
hcl

resource "azurerm_subnet_network_security_group_association" "apim_subnet_nsg_assoc" {
  subnet_id           = azurerm_subnet.apim_subnet.id
  network_security_group_id = azurerm_network_security_group.apim_subnet_nsg.id
}
```

Por que: As regras do NSG só funcionam quando associadas à subnet. É como instalar a portaria no condomínio.

10. Terraform State - Resource Group Separado

```
hcl
```

```
resource "azurerm_resource_group" "terraform_state_rg" {  
  name     = "rg-terraform-state"  
  location = "Brazil South"  
}
```

Por que separar?

- Se deletarmos `rg-tchungry-prod`, o state do Terraform continua seguro
- **Boas práticas:** State é crítico e deve ter ciclo de vida independente

11. Storage Account para Terraform State

```
hcl  
  
resource "azurerm_storage_account" "terraform_state_storage" {  
  name            = "tfstatetchungryale"  
  account_tier    = "Standard"  
  account_replication_type = "LRS"  
}  
  
resource "azurerm_storage_container" "terraform_state_container" {  
  name            = "tfstate"  
  container_access_type = "private"  
}
```

O que é: Armazenamento remoto do "tfstate" (arquivo que rastreia toda a infraestrutura).

Por que remoto?

- **Colaboração:** Múltiplos pipelines/desenvolvedores veem o mesmo state
- **Segurança:** State contém informações sensíveis (IPs, nomes)
- **Lock:** Previne execuções simultâneas que corromperiam o state

LRS (Locally Redundant Storage):

- 3 cópias dos dados no mesmo datacenter
- Suficiente para state file (não são dados críticos de clientes)



Outputs (Saídas)

Os outputs são **fundamentais** porque outros repositórios usam essas informações:

hcl

```
output "vnet_name" {  
  value = azurerm_virtual_network.vnet.name  
}  
  
output "aks_subnet_id" {  
  value = azurerm_subnet.aks_subnet.id  
}
```

Fluxo:

1. `infra-foundational` cria a VNet
2. Exporta o `vnet_name` como output
3. `infra-compute` lê esse output via **data source** ou **remote state**
4. Usa o `vnet_name` para criar o AKS dentro da VNet correta

Ordem de Execução

1. terraform init
↓
2. terraform plan (visualiza o que será criado)
↓
3. terraform apply (cria os recursos)
↓
4. Outputs ficam disponíveis para outros repos

Decisões de Arquitetura

Por que 4 subnets separadas?

Subnet	CIDR	Recursos	Justificativa
snet-aks	10.0.1.0/24	Kubernetes pods/services	Isolamento do cluster
snet-apps	10.0.2.0/24	App Gateway, Functions	Recursos de aplicação
snet-apim	10.0.3.0/24	API Management	Requisito Azure + segurança
snet-private-endpoints	10.0.4.0/24	Conexões privadas	Acesso privado a PaaS

Por que Brazil South?

- **Latência:** Menor latência para usuários brasileiros

- **Compliance:** Dados ficam em território brasileiro (LGPD)
 - **Custo:** Geralmente mais barato que regiões internacionais
-

✓ Resultado Final

Após executar este repositório, você tem:

- ✓ 1 Resource Group (`rg-tchungry-prod`)
 - ✓ 1 VNet privada com 65k IPs disponíveis
 - ✓ 4 Subnets isoladas para diferentes propósitos
 - ✓ 1 NSG protegendo o APIM com regras específicas
 - ✓ 1 Storage Account para state do Terraform (em RG separado)
 - ✓ Outputs prontos para serem consumidos pelos próximos repositórios
-

Próximo passo: Com a "fortaleza" construída, partimos para criar os "armazéns" (banco de dados e storage) no repositório `infra-data`.

Repositório 2: TechChallenge-infra-data

Objetivo

Este repositório cria a **camada de persistência** da aplicação: banco de dados SQL para dados transacionais e Storage Account para arquivos (imagens de produtos). É executado **após** o `infra-foundational`.

O que este repositório faz?

Analogia

Se o `infra-foundational` construiu o condomínio, agora estamos construindo os "armazéns":

- **Azure SQL Database** = Cofre de alta segurança (dados estruturados, transações)
 - **Azure Blob Storage** = Depósito de arquivos (imagens, documentos)
-

Estrutura de Arquivos

Backend Configuration

```
hcl
```

```
terraform {  
  backend "azurerm" {  
    resource_group_name = "rg-terraform-state"  
    storage_account_name = "tfstatetchungryale"  
    container_name      = "tfstate"  
    key                 = "infra-data.tfstate"  
  }  
}
```

O que é: Configuração do **remote state** (estado remoto).

Por que isso é CRÍTICO:

- O Terraform precisa saber **o que já existe** na Azure antes de criar/modificar recursos
- Sem state, o Terraform tentaria criar tudo do zero toda vez
- O state fica no Storage Account criado pelo `infra-foundational`
- **Lock automático:** Previne que 2 pipelines executem simultaneamente

Key: "infra-data.tfstate":

- Cada repositório tem seu próprio arquivo de state
- `infra-foundational.tfstate` (rede)
- `infra-data.tfstate` (banco + storage)
- `infra-compute.tfstate` (AKS + Functions)
- Isolamento: problema num state não afeta os outros

Data Sources

```
hcl  
  
data "azurerm_resource_group" "rg" {  
  name = "rg-tchungry-prod"  
}
```

O que é: Leitura de recursos que **já existem** (criados por outro repositório).

Por que usar data source em vez de criar novo RG?

- O RG já foi criado pelo `infra-foundational`
- Queremos **reutilizar** o mesmo RG (organização)

- Se tentássemos criar de novo, daria erro de conflito



1. Azure SQL Server

hcl

```
resource "azurerm_mssql_server" "sqlserver" {  
  name                = "sql-tchungrgy-hnaia0"  
  resource_group_name = data.azure_rm_resource_group.rg.name  
  location             = data.azure_rm_resource_group.rg.location  
  version              = "12.0"  
  administrator_login  = "admintchungrgy"  
  administrator_login_password = var.sql_admin_password  
  
  minimum_tls_version    = "1.2"  
  public_network_access_enabled = true  
}
```

O que é: Servidor SQL gerenciado pela Microsoft.

Decisões Técnicas:

Nome: sql-tchungrgy-hnaia0

- Sufixo aleatório (`hnaia0`) porque o nome deve ser **globalmente único** na Azure
- Não pode haver dois `sql-tchungrgy` em toda a Azure mundial

Version: "12.0"

- SQL Server 2014 compatível
- Suporta todas as features modernas que precisamos

minimum_tls_version: "1.2"

- **Segurança:** TLS 1.0 e 1.1 são vulneráveis
- PCI-DSS e LGPD exigem TLS 1.2+
- Protege dados em trânsito

public_network_access_enabled: true

- **Controverso!** Normalmente seria `false` em produção
- **Por que true aqui?**
 1. Facilita desenvolvimento/troubleshooting inicial

2. Firewall rules controlam acesso (próximo bloco)
3. Em produção real, usariamos Private Endpoint (subnet `snet-private-endpoints`)

Evolução recomendada:

```
hcl

# Fase atual: Público com firewall
public_network_access_enabled = true

# Produção: Privado via Private Endpoint
public_network_access_enabled = false
+ azure_rm_private_endpoint + link com snet-private-endpoints
```

2. Azure SQL Database

```
hcl

resource "azurerm_mssql_database" "sqlldb" {
  name      = "Hungry"
  server_id = azurerm_mssql_server.sqlserver.id
  sku_name  = "S0"

  collation      = "SQL_Latin1_General_CP1_CI_AS"
  max_size_gb    = 250
  zone_redundant = false
}
```

O que é: O banco de dados propriamente dito (tabelas, dados).

Decisões Técnicas:

SKU: "S0" (Standard)

Tier	vCores	RAM	IOPS	Custo	Uso
S0	Compartilhado	250MB	Baixo	~R\$60/mês	Dev/Test
S1	1	1GB	Médio	~R\$120/mês	Pequeno prod
P1	2	4GB	Alto	~R\$600/mês	Produção

Por que S0?

- Ambiente de desenvolvimento/apresentação
- Tráfego baixo esperado

- Fácil upgrade para S1/P1 sem downtime

Collation: SQL_Latin1_General_CP1_CI_AS

- **CI** = Case Insensitive ("João" = "joão")
- **AS** = Accent Sensitive ("São" ≠ "Sao")
- Padrão brasileiro, funciona bem com português

max_size_gb: 250

- Limite de tamanho do banco
- S0 suporta até 250GB
- Suficiente para aplicação de lanchonete

zone_redundant: false

- Se `true`, dados replicados em múltiplas zonas de disponibilidade
- **Custo:** 2-3x mais caro
- Para dev/demo, não justifica

3. SQL Firewall Rule

```
hcl

resource "azurerm_mssql_firewall_rule" "allow_azure_services" {
  name          = "AllowAllWindowsAzureIps"
  server_id     = azurerm_mssql_server.sqlserver.id
  start_ip_address = "0.0.0.0"
  end_ip_address  = "0.0.0.0"
}
```

O que é: Regra especial que permite serviços Azure acessarem o SQL.

IMPORTANTE: 0.0.0.0 não é "todo mundo"!

- `0.0.0.0` é uma **flag especial** da Azure
- Significa: "Permitir apenas serviços Azure internos"
- **AKS, Functions, APIM** conseguem acessar
- **Internet externa NÃO** consegue

Sem essa regra:

- ❌ AKS tenta conectar no SQL → BLOQUEADO
- ❌ Function tenta conectar no SQL → BLOQUEADO

Com essa regra:

- ✅ AKS (dentro da Azure) → SQL: OK
- ✅ Function (dentro da Azure) → SQL: OK
- ❌ Hacker da internet → SQL: BLOQUEADO

Segurança adicional possível:

```
hcl

# Permitir apenas subnet específica do AKS
resource "azurerm_mssql_virtual_network_rule" "aks_access" {
  name      = "AllowAKSSubnet"
  server_id = azurerm_mssql_server.sqlserver.id
  subnet_id = data.azurerm_subnet.aks_subnet.id
}
```



4. Storage Account

```
hcl

resource "azurerm_storage_account" "storage" {
  name                = "strgtchungryprod"
  resource_group_name = data.azurerm_resource_group.rg.name
  location            = data.azurerm_resource_group.rg.location
  account_tier        = "Standard"
  account_replication_type = "LRS"

  min_tls_version      = "TLS1_2"
  allow_nested_items_to_be_public = true

  blob_properties {
    delete_retention_policy {
      days = 7
    }
  }
}
```

O que é: Armazenamento de objetos (arquivos).

Por que usar Blob Storage em vez de guardar no banco?

1. **Performance:** Banco não é otimizado para binários grandes
2. **Custo:** Storage é ~10x mais barato que espaço em SQL
3. **Escalabilidade:** Suporta petabytes sem degradação
4. **CDN:** Pode servir imagens diretamente via HTTPS

Decisões Técnicas:

account_tier: "Standard"

Tier	Performance	Uso	Custo
Standard	HDD	Blobs, arquivos gerais	Baixo
Premium	SSD	VMs, discos high-IOPS	Alto

Para imagens de produtos, Standard é perfeito.

account_replication_type: "LRS"

Tipo	Cópias	Localização	Custo	Durabilidade
LRS	3	Mesmo datacenter	Mais barato	99.999999999% (11 noves)
GRS	6	Região secundária	2x LRS	99.9999999999999% (16 noves)

Por que LRS?

- 3 cópias já garantem durabilidade altíssima
- Se perdermos uma imagem de hambúrguer, não é catastrófico
- GRS só faz sentido para dados críticos de negócio

allow_nested_items_to_be_public: true

- Permite que containers/blobs sejam públicos
- **Necessário** para servir imagens de produtos via URL
- Exemplo: `https://strgtchungryprod.blob.core.windows.net/imagens/hamburger.jpg`

delete_retention_policy: 7 dias

- Se deletar um blob, ele fica "soft deleted" por 7 dias
 - **Proteção contra acidentes:** "Ops, deletei a foto errada!"
 - Após 7 dias, é deletado permanentemente
-

5. Storage Container (Imagens)

```
hcl

resource "azurerm_storage_container" "images" {
  name                = "imagens"
  storage_account_name = azurerm_storage_account.storage.name
  container_access_type = "blob"
}
```

O que é: "Pasta" dentro do Storage Account.

container_access_type: "blob"

Tipo	Acesso	Uso
private	Só com autenticação	Documentos sensíveis
blob	Leitura pública de blobs individuais	Imagens de produtos
container	Listagem pública de todos os blobs	Não recomendado

Por que "blob"?

- Cliente pode acessar `hamburger.jpg` via URL pública
- Cliente **NÃO** pode listar todos os arquivos do container
- Balanço entre usabilidade e segurança

Fluxo de uso:

1. Admin faz upload de "hamburger.jpg" via API
2. API salva no container "imagens"
3. API retorna URL: `https://strg.../imagens/hamburger.jpg`
4. Frontend exibe a imagem diretamente

Outputs

Connection String do SQL

```
hcl

output "db_connection_string" {
  value = "Server=tcp:${azurerm_mssql_server.sqlserver.fully_qualified_domain_name},1433;..."
  sensitive = true
}
```


sensitive = true: Não aparece nos logs do Terraform (contém senha).

Uso: Copiado para GitHub Secrets → Injetado na API e Functions.

Storage Connection String

```
hcl

output "storage_account_connection_string" {
  value     = azurerm_storage_account.storage.primary_connection_string
  sensitive = true
}
```

Uso: API usa para fazer upload de imagens.

Dependências

```
infra-foundational (cria RG)
    ↓
infra-data (usa RG existente)
```

O Terraform resolve automaticamente:

```
hcl

data "azurerm_resource_group" "rg" {
  name = "rg-tchungry-prod" # ← Criado por infra-foundational
}
```

Resultado Final

Após executar este repositório:

-  1 SQL Server gerenciado (`sql-tchungry-hnaia0`)
 -  1 SQL Database com 250GB (`Hungry`)
 -  Firewall configurado (só Azure Services)
 -  1 Storage Account (`strgtchungryprod`)
 -  1 Container público para imagens (`imagens`)
 -  Connection strings prontas para uso nos outros repos
-

Próximo passo: Criar os recursos de computação (AKS e Azure Functions) no repositório `infra-compute`.

Repositório 3: TechChallenge-infra-compute

Objetivo

Este repositório cria a **camada de computação**: onde o código da aplicação efetivamente roda. Inclui o cluster Kubernetes (AKS) para a API principal e uma Azure Function para autenticação serverless.

O que este repositório faz?

Analogia

Se `infra-foundational` construiu o condomínio e `infra-data` os armazéns, agora construímos as **fábricas** onde o trabalho acontece:

- **Azure Kubernetes Service (AKS)** = Fábrica principal com múltiplas linhas de produção (pods)
 - **Application Gateway Ingress Controller (AGIC)** = Porteiro interno que roteia requisições
 - **Azure Container Registry (ACR)** = Depósito de "moldes" (imagens Docker)
 - **Azure Function** = Oficina especializada pequena (autenticação serverless)
-

Data Sources (Leitura de Recursos Existentes)

```
hcl

data "azurerm_resource_group" "rg" {
  name = "rg-tchungrgy-prod"
}

data "azurerm_virtual_network" "vnet" {
  name = "vnet-tchungrgy-prod"
}

data "azurerm_subnet" "aks_subnet" {
  name = "snet-aks"
}
```

Por que tantos data sources?

- Este repo **não cria** a VNet nem as subnets
 - Ele **lê** o que foi criado por `infra-foundational`
 - Terraform valida que os recursos existem antes de prosseguir
-

1. Azure Container Registry (ACR)

```
hcl

resource "azurerm_container_registry" "acr" {
  name           = "acrthungryprod"
  resource_group_name = data.azurerm_resource_group.rg.name
  location       = data.azurerm_resource_group.rg.location
  sku            = "Basic"
  admin_enabled   = true
}
```

O que é: Registro privado de imagens Docker (tipo Docker Hub, mas privado).

Por que precisamos?

- GitHub Actions compila a API .NET → gera imagem Docker
- Imagem é enviada (push) para o ACR
- AKS puxa (pull) a imagem do ACR e roda nos pods

Fluxo:

Código .NET → Docker build → Push para ACR → AKS pull → Pod rodando

SKU: "Basic"

SKU	Storage	Throughput	Geo-replication	Custo/mês
Basic	10GB	Baixo	Não	~R\$25
Standard	100GB	Médio	Não	~R\$100
Premium	500GB	Alto	Sim	~R\$500+

Por que Basic?

- Aplicação pequena, poucas imagens (~500MB cada)
- 10GB = ~20 versões da imagem
- Upgrade para Standard é instantâneo se necessário

admin_enabled: true

- Habilita usuário/senha para o ACR
- Necessário para GitHub Actions fazer push
- Em produção enterprise, usaríamos **Service Principal** em vez de admin

2. Azure Kubernetes Service (AKS) com AGIC

hcl

```
resource "azurerm_kubernetes_cluster" "aks" {
  name          = "aks-tchungrgy-prod"
  location      = data.azurerm_resource_group.rg.location
  resource_group_name = data.azurerm_resource_group.rg.name
  dns_prefix    = "akstchungrgy"

  default_node_pool {
    name          = "default"
    node_count    = 1
    vm_size       = "Standard_E2s_v3"
    vnet_subnet_id = data.azurerm_subnet.aks_subnet.id
  }

  identity {
    type = "SystemAssigned"
  }

  network_profile {
    network_plugin = "azure"
    service_cidr   = "10.240.0.0/16"
    dns_service_ip = "10.240.0.10"
  }

  ingress_application_gateway {
    gateway_name = "agw-ingress-tchungrgy"
    subnet_id    = data.azurerm_subnet.app_subnet.id
  }
}
```

O que é: Cluster Kubernetes gerenciado pela Microsoft.

Decisões Técnicas Explicadas:

Node Pool: 1 node × Standard_E2s_v3

VM Size: Standard_E2s_v3

Série	vCPUs	RAM	Disco	Uso
B2s	2	4GB	HDD	Muito barato, baixa performance
E2s_v3	2	16GB	SSD	Memory-optimized
D2s_v3	2	8GB	SSD	Balanced

Por que E2s_v3?

- Nossa API .NET consome ~500MB-1GB de RAM por pod
- Com 16GB, podemos ter até **10-15 pods** no mesmo node
- SSD garante I/O rápido para logs e temp files
- **Custo:** ~R\$250/mês (24h/dia rodando)

Por que apenas 1 node?

- Ambiente de demonstração
- Produção real: mínimo 3 nodes (alta disponibilidade)
- Podemos adicionar nodes dinamicamente (scale out)

vnet_subnet_id: snet-aks

- Pods do AKS ganham IPs da subnet `10.0.1.0/24`
- **Integração nativa** com a VNet
- Pods podem acessar outros recursos da VNet (SQL, Functions) via IP privado

identity: SystemAssigned

```
hcl
identity {
  type = "SystemAssigned"
}
```

- AKS ganha uma **identidade gerenciada** automaticamente
- Não precisamos criar/gerenciar Service Principals manualmente
- Azure cuida de rotação de credenciais automaticamente
- Essa identidade é usada para:
 - Puxar imagens do ACR
 - Criar Load Balancers
 - Gerenciar Application Gateway

Network Plugin: "azure"

```
hcl

network_profile {
  network_plugin = "azure"
  service_cidr   = "10.240.0.0/16"
  dns_service_ip = "10.240.0.10"
}
```

Azure CNI vs Kubenet:

Plugin	IPs	Performance	Complexidade
kubenet	Pods usam IPs "fake", NAT para sair	Boa	Simples
azure	Pods ganham IP real da VNet	Melhor	Moderada

Por que Azure CNI?

- Pods têm IPs reais da subnet (10.0.1.X)
- **Comunicação direta** com outros recursos da VNet
- Necessário para integração com Application Gateway
- NSG/Firewall rules funcionam diretamente com IPs dos pods

service_cidr: 10.240.0.0/16

- Range SEPARADO para **ClusterIP Services** internos do K8s
- **Não conflita** com a VNet (10.0.0.0/16)
- Serviços como `kube-dns`, `api-service-tc` ganham IPs desse range

dns_service_ip: 10.240.0.10

- IP do CoreDNS (serviço DNS interno do K8s)
- Pods usam isso como DNS resolver
- Deve estar dentro do `service_cidr`

■ Application Gateway Ingress Controller (AGIC)

```
hcl
```

```
ingress_application_gateway {  
  gateway_name = "agw-ingress-tchungry"  
  subnet_id    = data.azure_rm_subnet.app_subnet.id  
}
```

O que é: Controlador que transforma Application Gateway em Ingress do Kubernetes.

Como funciona:

1. Você cria um recurso `Ingress` no K8s (YAML)
2. AGIC detecta o novo Ingress
3. AGIC **configura automaticamente** o Application Gateway
4. Application Gateway roteia tráfego para os pods

Fluxo de requisição:

Internet → App Gateway (10.0.2.X) → AGIC → Pod (10.0.1.X)

Por que usar Application Gateway?

- **Layer 7 (HTTP/HTTPS):** entende URLs, headers, métodos
- **WAF (Web Application Firewall):** protege contra ataques
- **SSL Termination:** lida com HTTPS, pods recebem HTTP simples
- **Integração nativa** com VNet e NSG

subnet_id: snet-apps

- Application Gateway precisa de subnet dedicada
- **Não pode** compartilhar com AKS ou APIM
- Gateway fica em `10.0.2.0/24`

Gateway Name: agw-ingress-tchungry

- O Terraform **cria** o Application Gateway automaticamente
 - Você não precisa criar o recurso `azurerm_application_gateway` manualmente
 - O AKS cria e gerencia o gateway para você
-

3. Permissões do AKS (RBAC)

3.1. AKS puxa imagens do ACR

hcl

```
resource "azurerm_role_assignment" "aks_pull_from_acr" {  
  scope      = azurerm_container_registry.acr.id  
  role_definition_name = "AcrPull"  
  principal_id    = azurerm_kubernetes_cluster.aks.kubelet_identity[0].object_id  
  skip_service_principal_aad_check = true  
}
```

O que faz: Permite que o **kubelet** (agente que roda nos nodes) puxe imagens do ACR.

kubelet_identity: Identidade gerenciada dos nodes (não do control plane).

Sem essa permissão:

yaml

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
api-deployment-xxx	0/1	ErrImagePull	0	2m

Com essa permissão:

yaml

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
api-deployment-xxx	1/1	Running	0	2m

3.2. AGIC gerencia a subnet do Application Gateway

hcl

```
resource "azurerm_role_assignment" "aks_agic_subnet_contributor" {  
  scope      = data.azurerm_subnet.app_subnet.id  
  role_definition_name = "Network Contributor"  
  principal_id    = azurerm_kubernetes_cluster.aks.ingress_application_gateway[0].ingress_application_gateway_identity[0].object_id  
}
```

O que faz: AGIC pode criar/modificar recursos de rede na subnet do Application Gateway.

Por que necessário:

- AGIC cria IPs privados para o gateway
 - AGIC associa NSG rules
 - AGIC configura rotas de rede
-

3.3. AGIC gerencia o Application Gateway

```
hcl

resource "azurerm_role_assignment" "aks_agic_gateway_contributor" {
  scope      = data.azure_rm_resource_group.rg.id
  role_definition_name = "Contributor"
  principal_id = azurerm_kubernetes_cluster.aks.ingress_application_gateway[0].ingress_application_gateway_identity[0].principal_id
}
```

O que faz: AGIC tem permissão total para modificar o Application Gateway.

O que AGIC configura:

- Backend pools (lista de IPs dos pods)
- HTTP settings (porta 80, health probes)
- Listeners (porta 443 HTTPS)
- Routing rules (qual URL vai para qual backend)

Exemplo prático:

```
yaml
```

```
# Você cria este Ingress no K8s
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-ingress
spec:
  rules:
  - host: api.internal.techchallenge
    http:
      paths:
      - path: /
        backend:
          service:
            name: api-service-tc
            port: 80
```

AGIC traduz isso em configuração do Application Gateway:

```
Listener: api.internal.techchallenge:80
↓
Backend pool: [10.0.1.11:8080, 10.0.1.12:8080] ← IPs dos pods
↓
Health probe: GET http://10.0.1.11:8080/
```

3.4. AGIC lê configurações da VNet

```
hcl
resource "azurerm_role_assignment" "aks_agic_vnet_reader" {
  scope      = data.azure_rm_virtual_network.vnet.id
  role_definition_name = "Reader"
  principal_id = azurerm_kubernetes_cluster.aks.ingress_application_gateway[0].ingress_application_gateway_identity[0]
}
```

O que faz: AGIC pode ler (mas não modificar) informações da VNet.

Por que necessário:

- AGIC precisa saber o range de IPs da VNet
- AGIC valida que os pods estão na mesma VNet
- AGIC configura rotas corretas

⚡ 4. Azure Function App (Serverless)

4.1. Storage Account para a Function

```
hcl

resource "azurerm_storage_account" "function_storage" {
  name           = "stauthrgtchungryprod"
  account_tier   = "Standard"
  account_replication_type = "LRS"
}
```

Por que Functions precisam de Storage Account?

- Armazena o código da função (arquivo .zip)
- Logs de execução
- Estado interno (triggers, queues)
- **Obrigatório**, mesmo para HTTP Functions

4.2. App Service Plan

```
hcl

resource "azurerm_service_plan" "function_plan" {
  name     = "plan-auth-serverless"
  os_type  = "Linux"
  sku_name = "B1"
}
```

SKU: B1 (Basic)

Plan	Tipo	vCPU	RAM	Custo/mês	Uso
Y1 (Consumption)	Serverless	Compartilhado	1.5GB	~R\$5 + execuções	Cargas esporádicas
B1 (Basic)	Always-On	1	1.75GB	~R\$60	Carga constante
P1V2 (Premium)	Always-On	1	3.5GB	~R\$350	Produção

Por que B1 em vez de Consumption (Y1)?

- **Consumption:** Cold start de 5-10 segundos (primeira chamada demora)
- **B1:** Always-on, resposta imediata
- Autenticação precisa ser rápida (UX)

- Custo fixo previsível

4.3. Function App PRIVADA

```
hcl

resource "azurerm_linux_function_app" "auth_function" {
  name                = "func-tchungry-auth"
  storage_account_name = azurerm_storage_account.function_storage.name
  storage_account_access_key = azurerm_storage_account.function_storage.primary_access_key
  service_plan_id      = azurerm_service_plan.function_plan.id

  # CRÍTICO: Function PRIVADA
  public_network_access_enabled = false

  site_config {
    application_stack {
      dotnet_version      = "8.0"
      use_dotnet_isolated_runtime = true
    }
  }

  app_settings = {
    "FUNCTIONS_WORKER_RUNTIME" = "dotnet-isolated"
  }
}
```

public_network_access_enabled: false

- **CRÍTICO!** A função é **totalmente privada**
- Internet não consegue acessar `func-tchungry-auth.azurewebsites.net`
- **Só o APIM** consegue acessar (via integração VNet)

Como funciona o deploy se é privada?

- GitHub Actions **temporariamente** abre acesso público
- Faz o deploy
- **Fecha** acesso de novo
- Veja o workflow da Function (próximo documento)

dotnet_version: "8.0"

- .NET 8 LTS (suporte até 2026)

- Performance superior ao .NET 6/7
- Startup mais rápido

use_dotnet_isolated_runtime: true

- Função roda em processo isolado
 - Maior flexibilidade de dependências
 - Recomendado pela Microsoft para .NET 6+
-



Outputs Importantes

ACR Login Server

```
hcl

output "acr_login_server" {
  value = azurerm_container_registry.acr.login_server
}

# Output: acrtchungryprod.azurecr.io
```

Uso: GitHub Actions usa para fazer `docker push acrtchungryprod.azurecr.io/lanchonete-api:abc123`

Function Hostname

```
hcl

output "function_app_default_hostname" {
  value = azurerm_linux_function_app.auth_function.default_hostname
}

# Output: func-tchungry-auth.azurewebsites.net
```

Uso: APIM usa como backend URL: `https://func-tchungry-auth.azurewebsites.net/api`

AKS Node Resource Group

```
hcl

output "aks_resource_group" {
  value = azurerm_kubernetes_cluster.aks.node_resource_group
}

# Output: MC_rg-tchungry-prod_aks-tchungry-prod_brazilsouth
```

O que é: Resource group **criado automaticamente** pela Azure que contém:

- VMs dos nodes

- Discos dos nodes
- **Application Gateway** (criado pelo AGIC)
- Load Balancer interno
- IPs públicos

Por que é importante:

- Para acessar o Application Gateway via CLI: `az network application-gateway show --name agw-ingress-tchungry --resource-group MC_...`
- Para ver logs do AGIC
- Para troubleshooting de rede

Dependências e Ordem

```
infra-foundational (VNet + Subnets)
  ↓
infra-data (SQL + Storage)
  ↓
infra-compute (AKS + ACR + Functions)
```

Resultado Final

Após executar este repositório:

-  1 Azure Container Registry para imagens Docker
-  1 Cluster AKS com 1 node (Standard_E2s_v3, 16GB RAM)
-  Application Gateway Ingress Controller (AGIC) configurado
-  4 Role Assignments (ACR pull, AGIC permissions)
-  1 Azure Function App privada para autenticação
-  1 Storage Account para a Function
-  Outputs prontos para uso nos pipelines de deploy

Próximo passo: Configurar o API Management (APIM) como porta de entrada única no repositório `infra-gateway`.

GitHub Actions - Deploy da Azure Function

Workflow: deploy-function.yml

Este workflow implementa uma **estratégia de segurança crítica**: a Function App é **privada por padrão**, mas precisa ser **temporariamente aberta** durante o deploy e **imediatamente fechada** depois.

Problema que este workflow resolve

Dilema de Segurança:

-  Queremos: Function 100% privada (só APIM acessa)
-  Problema: Deploy precisa de acesso público (GitHub Actions não está na VNet)

Solução implementada:

```
1. Function está PRIVADA (só APIM)
↓
2. Pipeline ABRE temporariamente (45s)
↓
3. Pipeline faz DEPLOY
↓
4. Pipeline FECHA imediatamente
↓
5. Function volta a estar PRIVADA
```

Estrutura do Workflow

Trigger

```
yaml
on:
  push:
    branches:
      - main
  workflow_dispatch:
```

- Executa automaticamente ao fazer push na `main`
 - `workflow_dispatch`: Permite execução manual via interface do GitHub
-

Steps Detalhados

Steps 1-4: Build da Aplicação .NET

yaml

```
- name: 'Setup .NET'
  uses: actions/setup-dotnet@v4
  with:
    dotnet-version: '8.0.x'

- name: 'Build and Publish'
  run: |
    dotnet publish --configuration Release --output ./publish --self-contained true -p:PublishReadyToRun=true
```

O que faz:

- Instala .NET 8 SDK
- Compila a aplicação em modo Release (otimizado)
- `--self-contained true`: Inclui runtime .NET no pacote (não depende de .NET instalado na Azure)
- `PublishReadyToRun=true`: Pré-compila assemblies para startup mais rápido

Output: Pasta `./publish` com todos os binários prontos

Step 6: REMOVE Restrições de Rede (CRÍTICO)

yaml


```
- name: 'Remove network restrictions temporarily'
uses: Azure/CLI@v1
with:
  inlineScript: |
    # Remove restrições do SITE PRINCIPAL
    az functionapp config access-restriction show \
      --name $FUNC_NAME \
      --resource-group $RG_NAME \
      --query "ipSecurityRestrictions[].name" -o tsv | \
    while read rule; do
      if [ ! -z "$rule" ] && [ "$rule" != "Allow" ]; then
        az functionapp config access-restriction remove \
          --name $FUNC_NAME \
          --resource-group $RG_NAME \
          --rule-name "$rule"
      fi
    done

    # Remove restrições do SCM SITE (Kudu/Deploy)
    # ... mesmo processo para --scm-site
```

O que é SCM Site?

- **SCM** = Source Control Management (Kudu)
- É o site de **deploy** da Function: `func-tchungry-auth.scm.azurewebsites.net`
- GitHub Actions faz upload do código através do SCM, não do site principal
- **CRÍTICO:** Se não liberar o SCM, o deploy falha com `403 Forbidden`

Por que remover TODAS as regras?

- Regras podem ter sido criadas em deploys anteriores
- Evita conflitos de prioridades
- Garante slate limpo antes de restaurar

Duas camadas de segurança removidas:

1. **Site principal** (`func-tchungry-auth.azurewebsites.net`): Para requisições de clientes
2. **SCM site** (`func-tchungry-auth.scm.azurewebsites.net`): Para deploys

Step 7: Aguarda Propagação (45 segundos)

yaml

```
- name: 'Wait for network propagation'
run: |
  echo "🕒 Waiting for network changes to propagate..."
  sleep 45
```

Por que esperar?

- Mudanças de rede na Azure não são instantâneas
- Pode levar 30-60s para propagar por todos os datacenters
- Sem essa espera, o deploy pode falhar com `timeout` ou `connection refused`

Step 8: Configura Variáveis de Ambiente

```
yaml

- name: 'Set Application Settings'
  uses: Azure/CLI@v1
  with:
    inlineScript: |
      # Connection String do Banco
      az webapp config connection-string set \
        --name ${{ env.AZURE_FUNCTIONAPP_NAME }} \
        --resource-group ${{ env.AZURE_RESOURCE_GROUP }} \
        --connection-string-type SQLAzure \
        --settings "Default=${{ secrets.DB_CONNECTION_STRING }}"

      # Configurações JWT
      az functionapp config appsettings set \
        --name ${{ env.AZURE_FUNCTIONAPP_NAME }} \
        --resource-group ${{ env.AZURE_RESOURCE_GROUP }} \
        --settings \
          "JWT__Key=${{ secrets.JWT_KEY }}" \
          "JWT__Issuer=${{ secrets.JWT_ISSUER }}" \
          "JWT__Audience=${{ secrets.JWT_AUDIENCE }}"
```

O que configura:

Connection String "Default"

```
Server=sql-tchungrgy-xxx.database.windows.net,1433;  
Initial Catalog=Hungry;  
User ID=admintchungrgy;  
Password=xxx;  
Encrypt=True;
```

- Usada pelo código da Function para acessar o SQL Database
- Tipo `SQLAzure`: otimizado para Azure SQL

JWT Settings

- `JWT__Key`: Chave secreta para assinar/validar tokens (mínimo 32 caracteres)
- `JWT__Issuer`: Quem emite o token (ex: `TechChallenge`)
- `JWT__Audience`: Para quem o token é destinado (ex: `TechChallenge.API`)

Por que `JWT__Key` em vez de `JWT:Key`?

- Azure Function usa `__` (dois underscores) como separador de seções
- `JWT__Key` → `JWT:Key` no código C#
- Compatível com `IConfiguration` do .NET

Step 9: Deploy da Function

```
yaml  
  
- name: 'Deploy to Azure Function App'  
  uses: Azure/functions-action@v1  
  with:  
    app-name: ${env.AZURE_FUNCTIONAPP_NAME}  
    package: './publish'
```

O que faz:

1. Compacta a pasta `./publish` em um `.zip`
2. Envia o `.zip` para o SCM site (`func-tchungrgy-auth.scm.azurewebsites.net`)
3. Kudu (engine de deploy da Azure) descompacta e instala
4. Reinicia a Function App automaticamente

Tempo de deploy: ~1-2 minutos

Step 10: RESTAURA Restrições de Rede (CRÍTICO)

yaml

- name: 'Restore network restrictions'

if: always() # ← EXECUTA SEMPRE, mesmo se steps anteriores falharem

uses: Azure/CLI@v1

with:

inlineScript: |

=== SITE PRINCIPAL ===

Libera APENAS a subnet do APIM

az functionapp config access-restriction add \

--name \$FUNC_NAME \

--resource-group \$RG_NAME \

--rule-name AllowAPIM \

--action Allow \

--vnet-name vnet-tchungry-prod \

--subnet snet-apim \

--priority 100

Bloqueia TODO o resto

az functionapp config access-restriction add \

--name \$FUNC_NAME \

--resource-group \$RG_NAME \

--rule-name DenyAllPublic \

--action Deny \

--ip-address 0.0.0.0/0 \

--priority 2147483647 # ← Prioridade máxima (última a ser avaliada)

=== SCM SITE (mesmas regras) ===

az functionapp config access-restriction add \

--name \$FUNC_NAME \

--resource-group \$RG_NAME \

--rule-name AllowAPIM_SCM \

--action Allow \

--vnet-name vnet-tchungry-prod \

--subnet snet-apim \

--scm-site \

--priority 100

az functionapp config access-restriction add \

--name \$FUNC_NAME \

--resource-group \$RG_NAME \

--rule-name DenyAllPublic_SCM \

--action Deny \

--ip-address 0.0.0.0/0 \

--priority 2147483647 \

--scm-site

Como funcionam as regras de acesso:

Prioridade das Regras

Prioridade	Regra	Ação	Descrição
100	AllowAPIM	Allow	Permite subnet 10.0.3.0/24 (APIM)
2147483647	DenyAllPublic	Deny	Bloqueia todo o resto (0.0.0.0/0)

Como a Azure avalia:

- Requisição chega de `10.0.3.5` (APIM)
 - ✓ Match com regra 100 (AllowAPIM) → **PERMITIDO**
- Requisição chega de `45.232.23.184` (Internet)
 - ✗ Não match com regra 100
 - ✗ Match com regra 2147483647 (DenyAllPublic) → **BLOQUEADO**

Por que priority 2147483647?

- É o valor **máximo** permitido pela Azure
- Garante que é a **última regra** avaliada (fallback)
- Qualquer outra regra com prioridade menor será avaliada primeiro

`if: always()`

yaml

`if: always()`

CRÍTICO para segurança!

- Executa **mesmo se o deploy falhar**
- Garante que a Function nunca fica aberta acidentalmente
- Sem isso, um erro no Step 9 deixaria a Function pública permanentemente

Cenários protegidos:

- ✓ Deploy bem-sucedido → Fecha
- ✓ Deploy falhou → Fecha mesmo assim
- ✓ Pipeline cancelado → Fecha mesmo assim
- ✓ Timeout → Fecha mesmo assim

Step 11: Logout do Azure

```
yaml
- name: 'Azure Logout'
  if: always()
  run: az logout
```

- Limpa credenciais do runner
- Boa prática de segurança
- `if: always()`: Executa mesmo se outros steps falharem

Arquitetura de Segurança Final

Estado Normal (99.9% do tempo):

Internet (45.232.23.184)



 BLOQUEADO

APIM (10.0.3.X)



 PERMITIDO → Function App

Durante Deploy (~2 minutos):

Internet (GitHub Actions)



 PERMITIDO (temporário)

[Deploy acontece]



 FECHADO automaticamente

Secrets Necessários

Configure no GitHub: `Settings → Secrets → Actions`

Secret	Valor	Uso
AZURE_CREDENTIALS	Service Principal JSON	Login no Azure
FUNCTION_APP_NAME	func-tchungry-auth	Nome da Function
DB_CONNECTION_STRING	Server=tcp:...	Conexão com SQL
JWT_KEY	Chave secreta (32+ chars)	Assinatura de tokens
JWT_ISSUER	TechChallenge	Emissor do token
JWT_AUDIENCE	TechChallenge.API	Audiência do token

✓ Validação Pós-Deploy

1. Verificar se Function está privada

```
bash

# Tentar acessar diretamente (deve falhar)
curl https://func-tchungry-auth.azurewebsites.net/api/auth
# Output esperado: 403 Forbidden

# Acessar via APIM (deve funcionar)
curl https://apim-tchungry-gateway-latest.azure-api.net/api/auth
# Output esperado: 200 OK
```

2. Verificar regras de acesso

```
bash

az functionapp config access-restriction show \
  --name func-tchungry-auth \
  --resource-group rg-tchungry-prod
```

Output esperado:

```
json
```



```
{
  "ipSecurityRestrictions": [
    {
      "name": "AllowAPIM",
      "priority": 100,
      "action": "Allow",
      "vnetSubnetResourceId": "/.../snet-apim"
    },
    {
      "name": "DenyAllPublic",
      "priority": 2147483647,
      "action": "Deny",
      "ipAddress": "0.0.0.0/0"
    }
  ]
}
```

Troubleshooting

Deploy falha com "403 Forbidden"

Causa: SCM site ainda está bloqueado

Solução: Aumentar o `sleep` de 45s para 60s no Step 7

Function fica pública após deploy

Causa: Step 10 falhou ou foi cancelado

Solução: Executar manualmente:

```
bash

az functionapp config access-restriction add \
  --name func-tchungry-auth \
  --resource-group rg-tchungry-prod \
  --rule-name DenyAllPublic \
  --action Deny \
  --ip-address 0.0.0.0/0 \
  --priority 2147483647
```

APIM não consegue acessar Function

Causa: APIM não está na subnet correta ou regra AllowAPIM configurada errada

Solução: Verificar se APIM está em `snet-apim` (10.0.3.0/24)

Próximo passo: Documentar o repositório `infra-gateway` (API Management).

GitHub Actions - Deploy da API no AKS

Workflow: `deploy-api-aks.yml`

Este workflow implementa o **CI/CD completo** da API .NET: desde o build da imagem Docker até a validação de health do Application Gateway.

O que este workflow faz?

Código .NET → Docker Image → ACR → AKS → AGIC → App Gateway → APIM → Internet

Passos principais:

1. Compila o código .NET em imagem Docker
 2. Envia a imagem para o Azure Container Registry (ACR)
 3. Deploy no Kubernetes (AKS) usando manifestos YAML
 4. Configura o Ingress Controller (AGIC)
 5. Valida que o Application Gateway está saudável
-

Environment Variables

yaml

env:

```
ACR_LOGIN_SERVER: ${ secrets.ACR_LOGIN_SERVER } # acrtchungryprod.azurecr.io
RESOURCE_GROUP: ${ secrets.RESOURCE_GROUP_NAME } # rg-tchungry-prod
AKS_CLUSTER_NAME: ${ secrets.AKS_CLUSTER_NAME } # aks-tchungry-prod
IMAGE_TAG: ${ github.sha } # commit hash (ex: abc123f)
```

IMAGE_TAG = github.sha

- Cada commit gera uma tag única
 - Permite rollback fácil: `kubectrl set image deployment/api-deployment-tc api=acr.../lanchonete-api:abc123f`
 - Histórico completo no ACR
-

Steps Detalhados

Steps 1-2: Checkout e Login

yaml

```
- name: 'Checkout code'
  uses: actions/checkout@v4

- name: 'Azure Login'
  uses: azure/login@v1
  with:
    creds: ${{ secrets.AZURE_CREDENTIALS }}
```

AZURE_CREDENTIALS: Service Principal JSON

json

```
{
  "clientId": "xxx",
  "clientSecret": "xxx",
  "subscriptionId": "xxx",
  "tenantId": "xxx"
}
```

Step 3: Login no ACR

yaml

```
- name: 'Docker Login to ACR'
  uses: azure/docker-login@v1
  with:
    login-server: ${{ env.ACR_LOGIN_SERVER }}
    username: ${{ fromJson(secrets.AZURE_CREDENTIALS).clientId }}
    password: ${{ fromJson(secrets.AZURE_CREDENTIALS).clientSecret }}
```

O que faz: Autentica o Docker CLI no ACR.

Por que usar clientId/clientSecret?

- O mesmo Service Principal que faz login na Azure é usado aqui
 - `fromJson()` extrai valores do JSON de AZURE_CREDENTIALS
 - Alternativa: usar `admin_enabled = true` no ACR (menos seguro)
-

Step 4: Build da Imagem Docker

```
yaml
- name: 'Build Docker image'
  run: |
    docker build -t ${{ env.ACR_LOGIN_SERVER }}/lanchonete-api:${{ env.IMAGE_TAG }} .
```

O que acontece:

1. Lê o `Dockerfile` na raiz do repositório
2. Executa os layers do Dockerfile (FROM, COPY, RUN, etc.)
3. Gera uma imagem com tag: `acrthungryprod.azurecr.io/lanchonete-api:abc123f`

Exemplo de Dockerfile típico (.NET 8):

```
dockerfile

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
WORKDIR /app
EXPOSE 8080

FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY ["src/API/API.csproj", "src/API/"]
RUN dotnet restore "src/API/API.csproj"
COPY . .
WORKDIR "/src/src/API"
RUN dotnet build "API.csproj" -c Release -o /app/build

FROM build AS publish
RUN dotnet publish "API.csproj" -c Release -o /app/publish

FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "API.dll"]
```

Step 5: Push para ACR

```
yaml
```

```
- name: 'Push Docker image to ACR'
run: |
  docker push ${{ env.ACR_LOGIN_SERVER }}/lanchonete-api:${{ env.IMAGE_TAG }}
```

O que faz:

- Envia a imagem para o registry privado
- Layers são comprimidas e enviadas (otimizado, só envia diferenças)
- Tempo: ~30-60 segundos (depende do tamanho da imagem)

No ACR após push:

```
Repositories:
lanchonete-api
Tags:
- abc123f (latest)
- def456g
- ghi789h
```

Step 6: Configura kubectl para AKS

```
yaml
- name: 'Set AKS context'
uses: azure/aks-set-context@v3
with:
  resource-group: ${{ env.RESOURCE_GROUP }}
  cluster-name: ${{ env.AKS_CLUSTER_NAME }}
```

O que faz:

- Baixa as credenciais do cluster AKS
- Configura `~/.kube/config` com as credenciais
- Todos os comandos `kubectl` subsequentes afetam este cluster

Equivalente manual:

```
bash
az aks get-credentials --resource-group rg-tchungry-prod --name aks-tchungry-prod
```

Step 6.1: Garante Integração AKS ↔ ACR

```
yaml

- name: 'Ensure AKS-ACR integration'
run: |
  az aks update --name ${{ env.AKS_CLUSTER_NAME }} \
    --resource-group ${{ env.RESOURCE_GROUP }} \
    --attach-acr ${{ env.ACR_LOGIN_SERVER }}
```

O que faz:

- Dá permissão `AcrPull` para a identidade do AKS
- Permite que os nodes puxem imagens sem senha
- **Idempotente**: roda toda vez, mas só cria a permissão se não existir

Sem isso:

```
yaml

kubectl describe pod api-xxx

Events:
  Failed to pull image "acr.../lanchonete-api:xxx": 401 Unauthorized
```

Com isso:

```
yaml

kubectl get pods

NAME             READY  STATUS   RESTARTS  AGE
api-deployment-xxx  1/1    Running  0         2m
```

Step 7: Cria/Atualiza Secrets do Kubernetes

```
yaml

- name: 'Create/Update Kubernetes secrets'
run: |
  kubectl create secret generic api-secrets \
    --from-literal=ConnectionStrings__Default='${{ secrets.DB_CONNECTION_STRING }}' \
    --from-literal=AZURE_STORAGE_CONNECTION_STRING='${{ secrets.AZURE_STORAGE_CONNECTION_STRING }}' \
    --dry-run=client -o yaml | kubectl apply -f -
```

O que faz:

- Cria um Secret chamado `api-secrets`
- Contém 2 chaves: connection string do SQL e do Storage
- `--dry-run=client -o yaml`: Gera YAML sem aplicar
- `kubectl apply -f -`: Aplica o YAML (cria ou atualiza)

Por que esse padrão?

- `kubectl create secret` falha se o secret já existe
- Com `--dry-run + apply`, é **idempotente** (pode rodar múltiplas vezes)

Como a API usa:

```
yaml

# 02-api-deployment.yaml
env:
- name: ConnectionStrings__Default
  valueFrom:
    secretKeyRef:
      name: api-secrets
      key: ConnectionStrings__Default
```

Step 8: Substitui Placeholders nos Manifestos

```
yaml

- name: 'Replace placeholders in manifests'
  run: |
    find ./k8s -type f -name "*.yaml" -exec sed -i "s|__ACR_LOGIN_SERVER__|${{ env.ACR_LOGIN_SERVER }}|g" {} +
    find ./k8s -type f -name "*.yaml" -exec sed -i "s|__IMAGE_TAG__|${{ env.IMAGE_TAG }}|g" {} +
```

O que faz:

- Procura todos os `.yaml` na pasta `k8s/`
- Substitui `__ACR_LOGIN_SERVER__` pelo valor real (ex: `acrthungryprod.azurecr.io`)
- Substitui `__IMAGE_TAG__` pelo commit hash (ex: `abc123f`)

Antes (02-api-deployment.yaml):

```
yaml
```

```
spec:
  containers:
  - name: api
    image: __ACR_LOGIN_SERVER__/lanchonete-api:__IMAGE_TAG__
```

Depois da substituição:

```
yaml

spec:
  containers:
  - name: api
    image: acrtchungryprod.azurecr.io/lanchonete-api:abc123f
```

Step 9: Deploy no AKS

```
yaml

- name: 'Deploy to AKS'
  run: |
    kubectl apply -f k8s/
```

O que faz:

- Aplica todos os manifestos da pasta `k8s/` em ordem alfabética:
 1. `01-api-configmap.yaml` (ConfigMap com configurações)
 2. `02-api-deployment.yaml` (Pods da API)
 3. `03-api-service.yaml` (ClusterIP Service)
 4. `04-api-hpa.yaml` (Horizontal Pod Autoscaler)
 5. `05-api-ingress.yaml` (Ingress para AGIC)

Kubernetes reconciliation:

- Se recursos já existem, são **atualizados**
- Deployment faz **rolling update** (zero downtime)
- Pods antigos só são deletados após novos ficarem healthy

Rolling update:

Estado inicial: Pod v1 (running)



Deploy v2: Pod v2 (starting), Pod v1 (running)



Pod v2 healthy: Pod v2 (running), Pod v1 (terminating)



Estado final: Pod v2 (running)

Step 10: Verifica Status do Deployment

yaml

```
- name: 'Verify deployment status'
```

```
run: |
```

```
kubectl rollout status deployment/api-deployment-tc --timeout=5m
```

O que faz:

- **Espera** até que o deployment esteja completo (até 5 minutos)
- Se timeout ou erro, o step falha

Saída esperada (sucesso):

```
Waiting for deployment "api-deployment-tc" rollout to finish: 0 of 1 updated replicas are available...  
deployment "api-deployment-tc" successfully rolled out
```

Saída esperada (falha):

```
Waiting for deployment "api-deployment-tc" rollout to finish: 0 of 1 updated replicas are available...  
error: deployment "api-deployment-tc" exceeded its progress deadline
```

Depois, coleta logs e eventos:

bash

```
kubectl get pods -l app=api-tc
```

```
kubectl logs <POD_NAME> --tail=50
```

```
kubectl describe pod <POD_NAME>
```

Útil para troubleshooting se algo der errado.

Step 11: Reinicia AGIC

```
yaml

- name: 'Restart AGIC to process Ingress'
  run: |
    kubectl delete pod -n kube-system -l app=ingress-appgw
    sleep 30
    kubectl wait --for=condition=ready pod -n kube-system -l app=ingress-appgw --timeout=120s
    sleep 90
```

Por que reiniciar o AGIC?

- AGIC às vezes não detecta mudanças no Ingress imediatamente
- **Bug conhecido** do AGIC: cache pode ficar stale
- Deletar o pod força reproprocessamento completo

Fluxo:

1. Deleta pod do AGIC (K8s cria um novo automaticamente)
2. Espera 30s para o novo pod iniciar
3. Aguarda pod ficar **Ready** (até 120s)
4. Espera 90s para AGIC reconfigurar o Application Gateway

Total: ~2 minutos de espera

Step 12: Verifica se Ingress Tem IP

```
yaml

- name: 'Verify Ingress has IP address'
  run: |
    kubectl get ingress api-ingress

    INGRESS_IP=$(kubectl get ingress api-ingress -o jsonpath='{.status.loadBalancer.ingress[0].ip}' 2>/dev/null || echo "")

    if [ -z "$INGRESS_IP" ]; then
      echo " ⚠️ WARNING: Ingress still has no IP address!"
      exit 1
    else
      echo " ✅ Ingress has IP: $INGRESS_IP"
    fi
```

O que verifica:

- O Ingress deve ter um IP atribuído pelo AGIC
- IP é o endereço privado do Application Gateway (ex: `10.0.2.5`)

Ingress sem IP:

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
api-ingress	azure-application-gateway	api.internal.techchallenge		80	5m

❌ **Problema:** AGIC não configurou o gateway

Ingress com IP:

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
api-ingress	azure-application-gateway	api.internal.techchallenge	10.0.2.5	80	5m

✅ **Sucesso:** AGIC configurou corretamente

Step 13: Verifica Health do Application Gateway

```
yaml
- name: 'Verify Application Gateway backend health'
  run: |
    BACKEND_HEALTH=$(az network application-gateway show-backend-health \
      --name agw-ingress-tchungry \
      --resource-group MC_${{ env.RESOURCE_GROUP }}_${{ env.AKS_CLUSTER_NAME }}_brazilsouth \
      --query "backendAddressPools[0].backendHttpSettingsCollection[0].servers[0].health" -o tsv)

    if [ "$BACKEND_HEALTH" == "Healthy" ]; then
      echo "✅ Backend is healthy! Deployment successful!"
    else
      echo "⚠️ Backend health: $BACKEND_HEALTH"
    fi
```

O que verifica:

- Application Gateway consegue fazer health check nos pods?
- Pods estão respondendo corretamente?

Resource Group: MC_XXX

- Application Gateway está no **node resource group** (não no RG principal)

- Formato: `MC_{RG}_{CLUSTER}_{REGION}`
- Exemplo: `MC_rg-tchungry-prod_aks-tchungry-prod_brazilsouth`

Health States possíveis:

Estado	Significado	Ação
Healthy	Pod respondendo 200 OK	✅ Tudo certo
Unhealthy	Pod respondendo 5xx ou timeout	❌ Verificar logs do pod
Unknown	Gateway não consegue alcançar pod	❌ Verificar rede/NSG
Draining	Pod em processo de remoção	🕒 Aguardar

Output esperado (sucesso):

```
json
{
  "backendAddressPools": [
    {
      "backendHttpSettingsCollection": [
        {
          "servers": [
            {
              "address": "10.0.1.11",
              "health": "Healthy",
              "healthProbeLog": "Success. Received 200 status code"
            }
          ]
        }
      ]
    }
  ]
}
```

 Secrets Necessários

Secret	Exemplo	Onde Pegar
<code>AZURE_CREDENTIALS</code>	Service Principal JSON	<code>az ad sp create-for-rbac</code>
<code>ACR_LOGIN_SERVER</code>	<code>acrтчhungryprod.azurecr.io</code>	Output do <code>infra-compute</code>
<code>RESOURCE_GROUP_NAME</code>	<code>rg-tchungry-prod</code>	Nome do RG
<code>AKS_CLUSTER_NAME</code>	<code>aks-tchungry-prod</code>	Nome do cluster
<code>DB_CONNECTION_STRING</code>	<code>Server=tcp:...</code>	Output do <code>infra-data</code>

Secret	Exemplo	Onde Pegar
AZURE_STORAGE_CONNECTION_STRING	DefaultEndpoints...	Output do infra-data

✅ Validação Pós-Deploy

1. Verificar pods rodando

```
bash

kubectl get pods -l app=api-tc
```

NAME	READY	STATUS	RESTARTS	AGE
api-deployment-tc-68545ffc95-v9r4g	1/1	Running	0	5m

2. Verificar Ingress configurado

```
bash

kubectl get ingress api-ingress
```

NAME	ADDRESS	HOSTS	PORTS	AGE
api-ingress	10.0.2.5	api.internal.techchallenge	80	5m

3. Testar health do Application Gateway

```
bash

az network application-gateway show-backend-health \
  --name agw-ingress-tchungrgy \
  --resource-group MC_rg-tchungrgy-prod_aks-tchungrgy-prod_brazilsouth \
  --query "backendAddressPools[0].backendHttpSettingsCollection[0].servers[0].health"
```

"Healthy"

4. Testar via APIM (quando configurado)

```
bash

curl https://apim-tchungrgy-gateway-latest.azure-api.net/api/products
```

```
[{"id":1,"name":"Hamburguer","price":15.90}]
```

Pod com status ImagePullBackOff

Causa: AKS não consegue puxar imagem do ACR

Solução:

```
bash
```

```
az aks update --name aks-tchungry-prod --resource-group rg-tchungry-prod --attach-acr acrtchungryprod
```

Ingress sem IP (ADDRESS vazio)

Causa: AGIC não processou o Ingress

Solução:

```
bash
```

```
kubectl delete pod -n kube-system -l app=ingress-appgw  
kubectl get ingress -w # aguardar IP aparecer
```

Backend Unhealthy no App Gateway

Causa: Pod não responde no health probe

Solução:

```
bash
```

```
kubectl logs <POD_NAME>  
kubectl describe pod <POD_NAME>  
# Verificar se a API está ouvindo na porta 8080
```

Pod em CrashLoopBackOff

Causa: Aplicação falha ao iniciar

Solução:

```
bash
```

```
kubectl logs <POD_NAME> --previous # ver logs do container que crashou  
# Comum: connection string do DB errada
```

Próximo passo: Documentar o repositório `infra-gateway` (Terraform do API Management).

Repositório 4: TechChallenge-infra-gateway

Objetivo

Este repositório cria a **porta de entrada única** da aplicação: o **API Management (APIM)**. Ele atua como gateway inteligente que roteia requisições entre a API no AKS e a Azure Function, aplicando segurança e rate limiting.

O que este repositório faz?

Analogia

Se toda a infraestrutura é um condomínio fechado:

- **APIM** = Portaria principal com recepcionista inteligente
 - **Private DNS Zone** = Mapa interno do condomínio
 - **Backends** = Endereços internos que a recepcionista conhece
 - **Políticas** = Regras de segurança e roteamento da portaria
-

Data Sources e Remote State

Lendo o estado do infra-compute

```
hcl

data "terraform_remote_state" "compute" {
  backend = "azurerm"
  config = {
    resource_group_name = "rg-terraform-state"
    storage_account_name = "tfstatetchungryale"
    container_name      = "tfstate"
    key                 = "infra-compute.tfstate"
  }
}
```

O que é: Lê os **outputs** do repositório `infra-compute`.

Por que necessário?

- Precisamos do hostname da Azure Function
- Precisamos do nome do Application Gateway
- Precisamos do Resource Group do AKS (MC_XXX)

Outputs consumidos:

```
hcl

data.terraform_remote_state.compute.outputs.function_app_default_hostname
# Output: func-tchungry-auth.azurewebsites.net

data.terraform_remote_state.compute.outputs.aks_resource_group
# Output: MC_rg-tchungry-prod_aks-tchungry-prod_brazilsouth
```

Lendo o Application Gateway criado pelo AGIC

```
hcl

data "azurerm_application_gateway" "aks_agic_gateway" {
  name           = "agw-ingress-tchungry"
  resource_group_name = data.terraform_remote_state.compute.outputs.aks_resource_group
  depends_on     = [data.terraform_remote_state.compute]
}
```

Por que ler em vez de criar?

- O Application Gateway é **criado automaticamente** pelo AKS quando habilitamos AGIC
 - Não controlamos diretamente via Terraform
 - Precisamos ler para pegar o IP público
-

Lendo o IP Público do Application Gateway

```
hcl

data "azurerm_public_ip" "app_gateway_pip" {
  name           = "agw-ingress-tchungry-appgwpip"
  resource_group_name = data.terraform_remote_state.compute.outputs.aks_resource_group
}
```

Nome fixo: `agw-ingress-tchungry-appgwpip`

- Padrão criado pelo AGIC: `{gateway_name}-appgwpip`
 - IP público usado para comunicação com a internet
-

1. API Management (APIM)

hcl

```
resource "azurerm_api_management" "apim" {
  name                        = "apim-tchungrgy-gateway-latest"
  resource_group_name       = data.azure_rm_resource_group.rg.name
  location                  = data.azure_rm_resource_group.rg.location
  publisher_name            = "TechChallenge"
  publisher_email           = "admin@techchallenge.com"
  sku_name                  = "Developer_1"

  virtual_network_type = "External"
  virtual_network_configuration {
    subnet_id = data.azure_rm_subnet.apim_subnet.id
  }
}
```

O que é: Gateway de API gerenciado pela Microsoft.

SKU: Developer_1

SKU	vCPU	Throughput	SLA	Custo/mês	Uso
Developer	1	Limitado	Sem SLA	~R\$240	Dev/Test
Basic	2	1000 req/s	99.9%	~R\$800	Pequeno prod
Standard	4	2500 req/s	99.9%	~R\$3.200	Médio prod
Premium	8+	4000+ req/s	99.99%	~R\$12.000+	Enterprise

Por que Developer?

- Custo baixo para ambiente de demonstração
- **Sem SLA:** Pode ter downtime (não para produção real)
- **Limitação:** 1 gateway unit, não escala
- Funcionalidades completas (políticas, backends, etc.)

virtual_network_type: "External"

Tipo	Descrição	Acesso Público	Acesso VNet
External	APIM em VNet, mas com IP público	✓ Sim	✓ Sim
Internal	APIM em VNet, sem IP público	✗ Não	✓ Sim
None	APIM fora da VNet	✓ Sim	✗ Não

Por que External?

- **Internet pode acessar:** `https://apim-tchungry-gateway-latest.azure-api.net`
- **APIM está na VNet:** Pode acessar recursos privados (Function, Application Gateway)
- **Melhor dos dois mundos:** Público + acesso a recursos privados

Fluxo:

```
Internet (público)
  ↓
APIM (subnet 10.0.3.X) ← IP público + IP privado
  ↓
Function (privada, subnet 10.0.3.X)
Application Gateway (privado, subnet 10.0.2.X)
```

virtual_network_configuration.subnet_id

```
hcl

subnet_id = data.azurearm_subnet.apim_subnet.id
```

- APIM fica na subnet dedicada `snet-apim` (10.0.3.0/24)
- NSG desta subnet já foi configurado no `infra-foundational`
- APIM ganha IP privado: ex. `10.0.3.5`

🔍 2. Private DNS Zone (DNS Interno)

```
hcl

resource "azurerm_private_dns_zone" "internal_api" {
  name                = "api.internal.techchallenge"
  resource_group_name = data.azurearm_resource_group.rg.name
}

resource "azurerm_private_dns_zone_virtual_network_link" "dns_vnet_link" {
  name                = "dns-link-to-vnet"
  private_dns_zone_name = azurerm_private_dns_zone.internal_api.name
  virtual_network_id   = data.azurearm_virtual_network.vnet.id
}
```

O que é: Zona DNS privada que funciona **apenas dentro da VNet**.

Por que necessário?

- APIM precisa resolver `api.internal.techchallenge` para acessar o Application Gateway
- DNS público não sabe sobre `api.internal.techchallenge`
- DNS privado só funciona dentro da VNet (recursos externos não conseguem resolver)

Fluxo de resolução:

1. APIM (10.0.3.5) quer acessar `http://api.internal.techchallenge`
2. Azure DNS resolver verifica Private DNS Zones linkadas à VNet
3. Encontra zona `"api.internal.techchallenge"`
4. Retorna o A record: `20.201.xxx.xxx` (IP público do App Gateway)
5. APIM acessa o Application Gateway

virtual_network_link:

- **Liga** a DNS Zone à VNet
- Sem isso, recursos na VNet não conseguem resolver o DNS
- Apenas VNets linkadas podem usar esta zona DNS

3. DNS A Record (Apontando para Application Gateway)

```
hcl

resource "azurerm_private_dns_a_record" "ingress_dns_record" {
  name           = "@"
  zone_name      = azurerm_private_dns_zone.internal_api.name
  resource_group_name = data.azurerm_resource_group.rg.name
  ttl            = 300

  records = [
    for config in data.azurerm_application_gateway.aks_agic_gateway.frontend_ip_configuration :
    data.azurerm_public_ip.app_gateway_pip.ip_address
    if config.public_ip_address_id != null
  ]

  depends_on = [data.azurerm_application_gateway.aks_agic_gateway]
}
```

name: `"@"`

- `@` representa o **root** da zona DNS

- Cria registro para `api.internal.techchallenge` (não `xxx.api.internal.techchallenge`)

TTL: 300

- Time To Live = 5 minutos
- Cache DNS expira após 5 minutos
- Se IP do gateway mudar, leva no máximo 5 min para propagar

records: IP público do Application Gateway

```
hcl

records = [data.azure_rm_public_ip.app_gateway_pip.ip_address]
# Exemplo: ["20.201.45.123"]
```

Por que usar IP público?

- Application Gateway criado pelo AGIC tem frontend com IP público
- APIM pode acessar via IP público **OU** privado (está na mesma VNet)
- Usar IP público simplifica a configuração

Resultado:

```
bash

# De qualquer recurso na VNet:
nslookup api.internal.techchallenge
# Output: 20.201.45.123 (IP público do App Gateway)
```

🔌 4. APIM Backends

Backend 1: AKS API (via Application Gateway)

```
hcl

resource "azurerm_api_management_backend" "api_aks_backend" {
  name                = "apiaksbackend-v1-1"
  resource_group_name = data.azure_rm_resource_group.rg.name
  api_management_name = azurerm_api_management.apim.name
  protocol            = "http"
  url                 = "http://api.internal.techchallenge/api"
  depends_on          = [azurerm_private_dns_a_record.ingress_dns_record]
}
```

O que é: Configuração de backend que aponta para a API principal no AKS.

url: `http://api.internal.techchallenge/api`

- **Protocolo:** HTTP (não HTTPS)
 - Application Gateway faz SSL termination
 - Entre APIM e App Gateway usamos HTTP simples
- **Hostname:** Resolve via Private DNS Zone
- **Path base:** `/api` é prefixo de todas as rotas

Fluxo de requisição:

```
APIM recebe: GET /api/products
↓
Backend URL: http://api.internal.techchallenge/api
↓
Requisição final: http://api.internal.techchallenge/api/products
↓
DNS resolve: 20.201.45.123 (App Gateway)
↓
App Gateway roteia para pod no AKS
```

Backend 2: Azure Function (Auth)

```
hcl

resource "azurerm_api_management_backend" "auth_function_backend" {
  name          = "authfunctionbackend-v1-1"
  resource_group_name = data.azurerm_resource_group.rg.name
  api_management_name = azurerm_api_management.apim.name
  protocol      = "http"
  url           = "https://func-tchungrgy-auth.azurewebsites.net/api"
  depends_on    = [data.terraform_remote_state.compute]
}
```

url: `https://func-tchungrgy-auth.azurewebsites.net/api`

- **Protocolo:** HTTPS (obrigatório para Azure Functions)
- **Hostname:** Obtido do remote state do `infra-compute`
- **Path base:** `/api` (padrão de Azure Functions)

Como APIM acessa Function privada?

- Function tem regra de acesso permitindo subnet `snet-apim` (10.0.3.0/24)
- APIM está nesta subnet → pode acessar
- Internet NÃO pode acessar (bloqueado por firewall rules)

Fluxo de requisição:

APIM recebe: POST /api/auth/login



Backend URL: https://func-tchungry-auth.azurewebsites.net/api



Requisição final: https://func-tchungry-auth.azurewebsites.net/api/auth/login



Function processa e retorna JWT token

5. API Configuration

hcl

```
resource "azurerm_api_management_api" "lanchonete_api" {  
  name          = "lanchonete-api"  
  resource_group_name = data.azurerm_resource_group.rg.name  
  api_management_name = azurerm_api_management.apim.name  
  revision      = "1"  
  display_name   = "API Hungry"  
  path           = "api"  
  protocols      = ["https"]  
  subscription_required = false  
  depends_on     = [azurerm_api_management.apim]  
}
```

path: "api"

- Base path para todas as operações desta API
- URL final: `https://apim-xxx.azure-api.net/api/*`

protocols: ["https"]

- Apenas HTTPS é permitido
- HTTP redirect para HTTPS automaticamente

subscription_required: false

- Não exige subscription key

- Em produção, deveria ser `true` para rastreamento e cotas
- Exemplo com subscription:

```
bash
```

```
curl -H "Ocp-Apim-Subscription-Key: abc123" https://apim-xxx.azure-api.net/api/products
```

Operações Catch-All

```
hcl
```

```
locals {  
  http_methods = ["GET", "POST", "PUT", "DELETE", "PATCH"]  
}  
  
resource "azurerm_api_management_api_operation" "catch_all" {  
  for_each = toset(local.http_methods)  
  
  operation_id      = "catch-all-${lower(each.value)}"  
  api_name          = azurerm_api_management_api.lanchonete_api.name  
  api_management_name = azurerm_api_management.apim.name  
  resource_group_name = data.azurerm_resource_group.rg.name  
  display_name      = "Catch-all ${each.value}"  
  method            = each.value  
  url_template       = "/*"  
  
  response {  
    status_code = 200  
  }  
}
```

O que faz: Cria 5 operações (uma para cada método HTTP) que capturam **todas** as requisições.

Por que catch-all (`/*`)?

- Não queremos definir manualmente cada endpoint (`/products`, `/orders`, etc.)
- Backend (API no AKS) já tem todas as rotas definidas
- APIM age como **proxy transparente**

Operações criadas:

1. `GET /*` → Captura qualquer GET
2. `POST /*` → Captura qualquer POST

3. `PUT /*` → Captura qualquer PUT
4. `DELETE /*` → Captura qualquer DELETE
5. `PATCH /*` → Captura qualquer PATCH

Exemplo:

```
bash
```

```
# Todas essas requisições são capturadas
```

```
GET /api/products
```

```
POST /api/orders
```

```
PUT /api/products/123
```

```
DELETE /api/orders/456
```

```
PATCH /api/users/789
```

6. Política de Roteamento (O Cérebro do APIM)

```
hcl
```



```

resource "azurerm_api_management_api_policy" "lanchonete_api_policy" {
  api_name           = azurerm_api_management_api.lanchonete_api.name
  api_management_name = azurerm_api_management.apim.name
  resource_group_name = data.azurerm_resource_group.rg.name

  xml_content = <<XML
<policies>
<inbound>
  <base />
  <rate-limit calls="100" renewal-period="60" />
  <choose>
    <when condition="@context.Request.Url.Path.Contains("auth") || context.Request.Url.Path.Contains("register")">
      <set-backend-service backend-id="authfunctionbackend-v1-1" />
    </when>
    <otherwise>
      <set-backend-service backend-id="apiaksbackend-v1-1" />
    </otherwise>
  </choose>
</inbound>
<backend>
  <base />
</backend>
<outbound>
  <base />
</outbound>
<on-error>
  <base />
</on-error>
</policies>
XML
}

```

O que é: Lógica XML que define **como** o APIM processa requisições.

Seções da Política:

1. `<inbound>` - Processamento da Requisição

Executado **antes** de encaminhar para o backend.

Rate Limit

xml

```
<rate-limit calls="100" renewal-period="60" />
```

- **Proteção contra DDoS e abuso**
- Máximo de **100 chamadas por 60 segundos** por IP
- Se exceder: retorna `429 Too Many Requests`

Exemplo:

IP 45.232.23.184:

00:00 - Chamada 1 

00:01 - Chamada 50 

00:30 - Chamada 100 

00:35 - Chamada 101  429 Too Many Requests

01:01 - Contador reseta, chamadas voltam a funcionar

Roteamento Condicional (Choose)

```
xml
<choose>
  <when condition="@context.Request.Url.Path.Contains("auth") || context.Request.Url.Path.Contains("register")">
    <set-backend-service backend-id="authfunctionbackend-v1-1" />
  </when>
  <otherwise>
    <set-backend-service backend-id="apiaksbackend-v1-1" />
  </otherwise>
</choose>
```

Como funciona:

Condição: URL contém `"auth"` OU `"register"`

```
csharp
context.Request.Url.Path.Contains("auth") || context.Request.Url.Path.Contains("register")
```

Exemplos de match (vai para Function):

-  `POST /api/auth/login` → Contém "auth"
-  `POST /api/register` → Contém "register"
-  `POST /api/auth/validate-token` → Contém "auth"

Exemplos de NÃO match (vai para AKS):

-  `GET /api/products` → Não contém "auth" nem "register"
-  `POST /api/orders` → Não contém "auth" nem "register"

- ❌ `PUT /api/categories/5` → Não contém "auth" nem "register"

Fluxo de decisão:

Requisição chega: POST /api/auth/login



Política verifica: Path contém "auth"? ✅ SIM



`<set-backend-service backend-id="authfunctionbackend-v1-1" />`



APIM encaminha para: <https://func-tchungry-auth.azurewebsites.net/api/auth/login>

Requisição chega: GET /api/products



Política verifica: Path contém "auth"? ❌ NÃO

Política verifica: Path contém "register"? ❌ NÃO



`<otherwise>` → `<set-backend-service backend-id="apiaksbackend-v1-1" />`



APIM encaminha para: <http://api.internal.techchallenge/api/products>

2. `<backend>` - Envio para Backend

xml

`<backend>`

`<base />`

`</backend>`

- `<base />`: Usa comportamento padrão (forward request)
- Aqui poderíamos adicionar retry logic, circuit breakers, etc.

3. `<outbound>` - Processamento da Resposta

xml

`<outbound>`

`<base />`

`</outbound>`

- Executado **após** receber resposta do backend

- Podemos modificar headers, transformar JSON, adicionar CORS, etc.

Exemplo de CORS (se necessário):

```
xml

<outbound>
  <base />
  <cors>
    <allowed-origins>
      <origin>https://techchallenge.com</origin>
    </allowed-origins>
    <allowed-methods>
      <method>GET</method>
      <method>POST</method>
    </allowed-methods>
  </cors>
</outbound>
```

4. <on-error> - Tratamento de Erros

```
xml

<on-error>
  <base />
</on-error>
```

- Executado se qualquer step anterior falhar
- Podemos retornar erros customizados, log erros, etc.

Fluxo Completo End-to-End

Cenário 1: Login (vai para Function)

1. Cliente:

POST <https://apim-tchungry-gateway-latest.azure-api.net/api/auth/login>

Body: {"username":"admin","password":"123"}

2. APIM - Política <inbound>:

- Rate limit: 1/100 chamadas nesta hora ☒
- Choose: Path contém "auth"? ☒ SIM
- Backend: authfunctionbackend-v1-1

3. APIM → Azure Function:

POST `https://func-tchungry-auth.azurewebsites.net/api/auth/login`
(APIM na subnet 10.0.3.X pode acessar Function privada)

4. Function:

- Valida credenciais no SQL
- Gera JWT token
- Retorna: `{"token":"eyJhbGc..."}`

5. APIM - Política <outbound>:

- Passa resposta sem modificações

6. Cliente recebe:

200 OK

`{"token":"eyJhbGc..."}`

Cenário 2: Listar Produtos (vai para AKS)

1. Cliente:

GET `https://apim-tchungry-gateway-latest.azure-api.net/api/products`

2. APIM - Política <inbound>:

- Rate limit: 2/100 chamadas 
- Choose: Path contém "auth"?  NÃO
- Choose: Path contém "register"?  NÃO
- Backend: `apiaksbackend-v1-1`

3. APIM → Application Gateway:

GET `http://api.internal.techchallenge/api/products`
DNS resolve: 20.201.45.123

4. Application Gateway → Service → Pod:

GET `http://10.0.1.11:8080/api/products`

5. Pod (API .NET):

- Consulta SQL Database
- Retorna: `[{"id":1,"name":"Hamburguer"}]`

6. Resposta volta: Pod → Service → App Gateway → APIM → Cliente

Resultado Final

Após executar este repositório:

- ✓ 1 API Management com IP público e integração VNet
 - ✓ 1 Private DNS Zone para resolução interna
 - ✓ 1 DNS A Record apontando para Application Gateway
 - ✓ 2 Backends configurados (AKS e Function)
 - ✓ 1 API com operações catch-all
 - ✓ Políticas de rate limiting e roteamento inteligente
 - ✓ **Porta de entrada única** para toda a aplicação
-

Arquitetura Completa Finalizada! 🎉

Todos os 4 repositórios de infraestrutura estão documentados.

Resumo Executivo - Arquitetura TechChallenge Fase 3

Visão Geral

Transformamos uma API monolítica em uma **arquitetura cloud-native** na Azure, implementando segurança em camadas, automação total via CI/CD e escalabilidade horizontal.

Arquitetura em 4 Camadas

1 Camada de Rede (Fundação)

Repositório: `infra-foundational`

O que cria:

- 1 VNet privada (10.0.0.0/16) - 65k IPs disponíveis
- 4 Subnets isoladas por função
- NSG com regras de firewall para APIM
- Storage Account para Terraform State remoto

Decisão chave: Isolamento total via VNet garante que nenhum recurso fica exposto diretamente na internet.

2 Camada de Dados (Persistência)

Repositório: `infra-data`

O que cria:

- Azure SQL Database (SKU S0) - Dados transacionais

- Blob Storage (LRS) - Imagens de produtos
- Firewall rules permitindo apenas serviços Azure

Decisão chave: SQL gerenciado elimina gerenciamento de servidor, Blob Storage separa arquivos do banco (performance + custo).

3 Camada de Computação (Processamento)

Repositório: `infra-compute`

O que cria:

- Azure Kubernetes Service (1 node, 16GB RAM)
- Application Gateway Ingress Controller (AGIC)
- Azure Container Registry (ACR)
- Azure Function App (autenticação serverless)
- 4 Role Assignments (permissões RBAC)

Decisões chave:

- AKS com Azure CNI: Pods têm IPs reais da VNet
 - AGIC: Ingress nativo Azure, integração perfeita
 - Function privada: Só APIM pode acessar
 - Standard_E2s_v3: Memory-optimized para .NET
-

4 Camada de Gateway (Porta de Entrada)

Repositório: `infra-gateway`

O que cria:

- API Management (SKU Developer)
- Private DNS Zone para resolução interna
- 2 Backends (AKS via App Gateway + Azure Function)
- Políticas de rate limiting e roteamento inteligente

Decisão chave: APIM como **ponto único de entrada** simplifica segurança, observabilidade e versionamento.

Arquitetura de Segurança

Princípio: Zero Trust + Defense in Depth



Camadas de Proteção:

1. **APIM:** Rate limiting, autenticação, políticas
 2. **VNet:** Isolamento de rede, subnets dedicadas
 3. **NSG:** Firewall de camada 4
 4. **App Gateway:** WAF (Web Application Firewall)
 5. **SQL Firewall:** Só Azure Services
 6. **Function Access Rules:** Só subnet do APIM
-

Fluxo de Requisição Completo

Exemplo: GET /api/products

1. Cliente (Browser/Mobile)

↓ HTTPS

GET https://apim-tchungry-gateway-latest.azure-api.net/api/products

2. APIM (Inbound Policy)

✓ Rate limit check (1/100 requests)

✓ Path analysis: Contém "auth"? ✗ NÃO

✓ Backend: apiaksbackend-v1-1

↓ HTTP

GET http://api.internal.techchallenge/api/products

3. Private DNS Resolution

api.internal.techchallenge → 20.201.45.123 (App Gateway IP)

4. Application Gateway (10.0.2.5)

✓ SSL termination (HTTPS → HTTP)

✓ Ingress rules: Host match ✓, Path match ✓

✓ Backend pool: api-service-tc (ClusterIP)

↓ HTTP

GET http://10.240.0.15:80/api/products (Service ClusterIP)

5. Kubernetes Service (Load Balancer)

✓ Selector: app=api-tc

✓ Escolhe pod saudável (round-robin)

↓ HTTP

GET http://10.0.1.11:8080/api/products (Pod IP)

6. Pod (Container .NET 8)

✓ API recebe requisição

- ✓ Consulta SQL: `SELECT * FROM Products`
- ✓ Retorna JSON: `[{"id":1,"name":"Hamburguer","price":15.90}]`

7. Resposta volta (mesmo caminho reverso)

Pod → Service → App Gateway → APIM → Cliente

Tempo total: ~50-150ms (dependendo da query SQL)

Automação CI/CD

6 Pipelines GitHub Actions

Pipeline	Trigger	O que faz	Tempo
infra-foundational	Push main	Cria VNet, Subnets, NSG	~3 min
infra-data	Push main	Cria SQL, Blob Storage	~5 min
infra-compute	Push main	Cria AKS, ACR, Function	~15 min
infra-gateway	Push main	Cria APIM, DNS, Backends	~30 min
deploy-function	Push main	Deploy Azure Function (com abertura/fechamento de rede)	~2 min
deploy-api-aks	Push main	Build Docker → ACR → K8s → Validação	~5 min

Total para infraestrutura completa do zero: ~60 minutos

Escalabilidade e Performance

Escalonamento Horizontal Automático

Horizontal Pod Autoscaler (HPA):

```
yaml

minReplicas: 1
maxReplicas: 9
targetCPU: 60%
```

Cenário de carga:

Tempo	Requisições/s	CPU Média	Pods Ativos	Ação
10:00	10	40%	1	Normal
10:30	50	75%	1 → 2	HPA scale up
11:00	100	80%	2 → 3	HPA scale up
11:30	30	45%	3	Cooldown (5 min)
11:40	20	30%	3 → 2	HPA scale down

Capacidade máxima:

- 9 pods × 250m CPU limit = 2250m (2.25 CPUs)
- Node tem 2 CPUs disponíveis
- **Solução:** Adicionar mais nodes ao AKS

💰 Estimativa de Custos (Brazil South)

Custos Mensais Aproximados

Recurso	SKU	Custo/mês
AKS	1× Standard_E2s_v3	R\$ 250
SQL Database	S0 (250GB)	R\$ 60
Blob Storage	Standard LRS	R\$ 10
ACR	Basic	R\$ 25
Function App	B1 Plan	R\$ 60
APIM	Developer	R\$ 240
Application Gateway	Standard_v2	R\$ 300
Public IPs	2× Standard	R\$ 20
VNet/Subnets	Grátis	R\$ 0
Private DNS	Grátis (até 25 zones)	R\$ 0
TOTAL		~R\$ 965/mês

Otimizações para produção:

- APIM: Mudar para Basic (~R\$ 800) quando sair de dev
- SQL: Escalar para S1 ou P1 conforme carga
- AKS: Adicionar nodes com autoscaling
- App Gateway: Considerar WAF_v2 para segurança extra

Conceitos Aplicados

Kubernetes (AKS)

- ✓ Pods, ReplicaSets, Deployments
- ✓ Services (ClusterIP)
- ✓ Ingress (AGIC)
- ✓ ConfigMaps e Secrets
- ✓ Horizontal Pod Autoscaler (HPA)
- ✓ Resource Requests e Limits
- ✓ Azure CNI Networking

Azure Services

- ✓ Virtual Network (VNet) com subnets
- ✓ Network Security Groups (NSG)
- ✓ Private DNS Zones
- ✓ Azure Kubernetes Service (AKS)
- ✓ Application Gateway Ingress Controller (AGIC)
- ✓ Azure Container Registry (ACR)
- ✓ Azure Functions (Serverless)
- ✓ API Management (APIM)
- ✓ Azure SQL Database
- ✓ Blob Storage

DevOps e IaC

- ✓ Terraform (Infrastructure as Code)
- ✓ Remote State com locking
- ✓ GitHub Actions (CI/CD)
- ✓ Docker e Containerização
- ✓ GitOps workflow
- ✓ Secrets Management

Segurança

- ✓ Zero Trust Architecture
- ✓ Defense in Depth
- ✓ Private Endpoints
- ✓ Network Segmentation
- ✓ RBAC (Role-Based Access Control)
- ✓ Managed Identities
- ✓ Rate Limiting
- ✓ SSL/TLS Termination

Próximos Passos (Melhorias Futuras)

Observabilidade

- ☐ Application Insights para logs e métricas
- ☐ Log Analytics Workspace
- ☐ Prometheus + Grafana no AKS
- ☐ Distributed tracing (OpenTelemetry)

Segurança Avançada

- ☐ Azure Key Vault para secrets
- ☐ Private Endpoints para SQL e Storage
- ☐ WAF no Application Gateway
- ☐ Azure Policy para compliance
- ☐ Network Policies no Kubernetes

Alta Disponibilidade

- ☐ Multi-zone deployment (3 AZs)
- ☐ 3+ nodes no AKS
- ☐ Geo-replication do SQL (GRS)
- ☐ Traffic Manager para multi-region
- ☐ Backup automático do SQL

Performance

- ☐ Azure Redis Cache
- ☐ CDN para Blob Storage (imagens)
- ☐ Connection pooling otimizado
- ☐ Query optimization (SQL indexes)

DevOps

- ☐ Ambientes múltiplos (dev/staging/prod)
- ☐ Blue-Green deployments
- ☐ Canary releases
- ☐ Automated rollback
- ☐ Chaos engineering (Chaos Mesh)

Repositórios e Organização

```
|
|
|— infra-foundational/      # VNet, Subnets, NSG
|   |— main.tf
|   |— variables.tf
|   |— outputs.tf
|   |— .github/workflows/deploy.yml
|
|— infra-data/             # SQL, Storage
|   |— main.tf
|   |— variables.tf
|   |— outputs.tf
|   |— .github/workflows/deploy.yml
|
|— infra-compute/          # AKS, ACR, Functions
|   |— main.tf
|   |— variables.tf
|   |— outputs.tf
|   |— .github/workflows/deploy.yml
|
|— infra-gateway/          # APIM, DNS
|   |— main.tf
|   |— variables.tf
|   |— outputs.tf
|   |— .github/workflows/deploy.yml
|
|— TechChallenge/          # API .NET principal
|   |— src/
|   |— Dockerfile
|   |— k8s/
|   |   |— 01-api-configmap.yaml
|   |   |— 02-api-deployment.yaml
|   |   |— 03-api-service.yaml
|   |   |— 04-api-hpa.yaml
|   |   |— 05-api-ingress.yaml
|   |— .github/workflows/deploy-aks.yml
|
|— TechChallenge-app-function/ # Azure Function (Auth)
|   |— src/
|   |— host.json
|   |— .github/workflows/deploy-function.yml
```

✅ Checklist de Validação Final

Infraestrutura

- ✅ VNet criada com 4 subnets
- ✅ NSG configurado para APIM
- ✅ SQL Database provisionado e acessível
- ✅ Blob Storage com container "imagens"
- ✅ AKS rodando com 1 node
- ✅ ACR criado e integrado com AKS
- ✅ Function App privada
- ✅ APIM configurado com backends

Segurança

- ✅ Function acessível apenas via APIM
- ✅ SQL com firewall (só Azure Services)
- ✅ Secrets no Kubernetes (não hardcoded)
- ✅ Rate limiting no APIM (100 req/min)
- ✅ HTTPS obrigatório no APIM
- ✅ Managed Identities (sem senhas)

CI/CD

- ✅ 6 pipelines funcionando
- ✅ Secrets configurados no GitHub
- ✅ Deploy automático no push main
- ✅ Validação de health checks
- ✅ Rollback automático em caso de falha

Aplicação

- ✅ API rodando no AKS
- ✅ Pods com status Running
- ✅ Ingress com IP atribuído
- ✅ App Gateway backend Healthy
- ✅ Function respondendo via APIM
- ✅ HPA configurado e funcional

Testes End-to-End

- ✅ `GET /api/products` retorna 200
- ✅ `POST /api/auth/login` retorna JWT
- ✅ `POST /api/orders` cria pedido no SQL
- ✅ Upload de imagem salva no Blob

Conquistas desta Arquitetura

✓ Cloud-Native

- Containers, orquestração, serverless
- Escalabilidade horizontal automática
- Alta disponibilidade via Kubernetes

✓ Segura por Design

- Zero Trust: nada exposto diretamente
- Defense in Depth: múltiplas camadas
- Secrets gerenciados, não hardcoded

✓ 100% Automatizada

- Infrastructure as Code (Terraform)
- CI/CD end-to-end (GitHub Actions)
- Zero intervenção manual

✓ Observável

- Logs centralizados
- Health checks em todas as camadas
- Métricas de performance (HPA)

✓ Escalável

- De 1 a 9 pods automaticamente
- Adicionar nodes é trivial
- Multi-region possível

✓ Custo-Otimizada

- SKUs apropriados para cada recurso
 - Escalamento baseado em demanda
 - Serverless para auth (paga por uso)
-

Aprendizados e Boas Práticas

1. Separe infraestrutura em repositórios por responsabilidade

- Facilita manutenção
- Deploy independente
- Blast radius limitado

2. Use Remote State com locking

- Evita conflitos
- Permite colaboração
- Histórico de mudanças

3. Private por padrão, público por exceção

- Minimize superfície de ataque
- Use VNet Integration
- Private Endpoints quando possível

4. Automatize tudo

- Infraestrutura como código
- Testes automatizados
- Deploy em um clique

5. Pense em observabilidade desde o início

- Logs estruturados
- Health checks
- Métricas de negócio



Conclusão

Esta arquitetura demonstra **nível profissional** de engenharia de software:

-  **Escalável:** HPA de 1-9 pods, pode adicionar mais nodes
-  **Segura:** Zero Trust, Defense in Depth, secrets gerenciados
-  **Confiável:** HA via Kubernetes, health checks, rollback automático
-  **Automatizada:** 100% IaC, CI/CD completo, zero clicks
-  **Observável:** Logs, métricas, health probes em todas as camadas

-  **Manutenível:** Código organizado, documentado, versionado

De uma API monolítica para uma arquitetura cloud-native enterprise-grade! 🚀

Contatos e Recursos

- **Documentação Azure:** <https://docs.microsoft.com/azure>
 - **Terraform Registry:** <https://registry.terraform.io/providers/hashicorp/azurerms>
 - **Kubernetes Docs:** <https://kubernetes.io/docs>
 - **GitHub Actions:** <https://docs.github.com/actions>
-

Documentação criada em: Outubro 2025

Versão: 1.0

Status:  Completa e Validada

Manifestos Kubernetes (k8s/)

Estes arquivos definem **como a API roda no Kubernetes**. São aplicados pelo pipeline de deploy e gerenciam pods, serviços, escalonamento e roteamento.

01-api-configmap.yaml

```
yaml

apiVersion: v1
kind: ConfigMap
metadata:
  name: api-configmap-tc
data:
  ASPNETCORE_ENVIRONMENT: "Development"
  Logging__LogLevel__Default: "Information"
```

O que é: Armazena configurações não-sensíveis como variáveis de ambiente.

ASPNETCORE_ENVIRONMENT: "Development"

- Define o ambiente da aplicação .NET
- **Development:** Mostra erros detalhados, Swagger habilitado
- **Production:** Erros genéricos, sem Swagger

⚠️ **Recomendação:** Mudar para "Production" em ambiente real:

```
yaml
```

```
ASPNETCORE_ENVIRONMENT: "Production"
```

Logging__LogLevel__Default: "Information"

- Nível de log: `Trace` < `Debug` < `Information` < `Warning` < `Error` < `Critical`
- `Information`: Logs operacionais normais (requests HTTP, queries SQL)
- Equivalente a `appsettings.json`:

```
json
```

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information"
    }
  }
}
```

Como a API usa:

```
yaml
```

```
# 02-api-deployment.yaml
envFrom:
- configMapRef:
    name: api-configmap-tc
```

Todas as chaves do ConfigMap viram variáveis de ambiente no container.

🚀 02-api-deployment.yaml

```
yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment-tc
spec:
  replicas: 1
  selector:
    matchLabels:
      app: api-tc
  template:
    metadata:
      labels:
        app: api-tc
    spec:
      containers:
        - name: api-container-tc
          image: __ACR_LOGIN_SERVER__/lanchonete-api: __IMAGE_TAG__
          imagePullPolicy: Always
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: "100m"
            limits:
              cpu: "250m"
          envFrom:
            - secretRef:
                name: api-secrets
            - configMapRef:
                name: api-configmap-tc
```

O que é: Define como os **Pods** (instâncias da API) devem ser criados e gerenciados.

replicas: 1

- Quantas cópias da API rodam simultaneamente
- ① = ambiente de dev/demo
- Produção: mínimo ③ (alta disponibilidade)

Mudança em runtime:

```
bash
```

```
kubectl scale deployment api-deployment-tc --replicas=3
```

selector.matchLabels: app=api-tc

- Como o Deployment identifica seus pods
- Todos os pods gerenciados por este Deployment têm label `app=api-tc`

image: ACR_LOGIN_SERVER/lanchonete-api:IMAGE_TAG

- Placeholders substituídos pelo pipeline:
 - `ACR_LOGIN_SERVER` → `acrchungryprod.azurecr.io`
 - `IMAGE_TAG` → `abc123f` (commit hash)
- Final: `acrchungryprod.azurecr.io/lanchonete-api:abc123f`

imagePullPolicy: Always

- **Always:** K8s sempre puxa a imagem do ACR (mesmo se já existe localmente)
- **IfNotPresent:** Usa cache local se existir
- **Never:** Nunca puxa, só usa local

Por que Always?

- Garante que sempre usa a versão mais recente
- Evita bugs de "funcionava localmente mas não no cluster"

containerPort: 8080

- A aplicação .NET **escuta** na porta 8080 dentro do container
- Definido no `Program.cs` ou `launchSettings.json`
- **Não expõe** publicamente (isso é feito pelo Service)

resources.requests.cpu: "100m"

- **Request:** Quantidade **garantida** de CPU para o pod
- `100m` = 0.1 core = 10% de 1 CPU
- Kubernetes **reserva** esse recurso no node
- Pod só é agendado se o node tiver 100m disponível

resources.limits.cpu: "250m"

- **Limit:** Quantidade **máxima** de CPU que o pod pode usar
- `250m` = 0.25 core = 25% de 1 CPU
- Se pod tentar usar mais, é **throttled** (não crashado)

Exemplo prático:

Node com 2 CPUs (2000m):

- Pod 1: request=100m, limit=250m
- Pod 2: request=100m, limit=250m
- Pod 3: request=100m, limit=250m

Total reservado: 300m (15% do node)

Total possível: 750m se todos usarem no máximo

Por que separar request e limit?

- **Request baixo:** Empacota mais pods no mesmo node (economia)
- **Limit alto:** Permite bursts de CPU quando necessário

envFrom: secretRef + configMapRef

yaml

envFrom:

- secretRef:
name: api-secrets
- configMapRef:
name: api-configmap-tc

O que faz: Injeta **todas** as chaves como variáveis de ambiente.

api-secrets contém:

- `ConnectionStrings__Default` → Vira env var no container
- `AZURE_STORAGE_CONNECTION_STRING` → Vira env var no container

api-configmap-tc contém:

- `ASPNETCORE_ENVIRONMENT` → Vira env var no container
- `Logging_LogLevel_Default` → Vira env var no container

No código C#:

csharp

```
// Lê ConnectionStrings__Default automaticamente
services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(Configuration.GetConnectionString("Default"));
```

03-api-service.yaml

```
yaml

apiVersion: v1
kind: Service
metadata:
  name: api-service-tc
spec:
  type: ClusterIP
  selector:
    app: api-tc
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

O que é: Expõe os pods como um **endpoint estável** dentro do cluster.

type: ClusterIP

Tipo	Exposto	Uso
ClusterIP	Só dentro do cluster	APIs internas, backends
NodePort	Node IP:porta	Desenvolvimento local
LoadBalancer	IP público via LB	Aplicações públicas

Por que ClusterIP?

- API é **privada**, não deve ser acessível diretamente da internet
- **Ingress/AGIC** acessa via ClusterIP
- **APIM** acessa via Application Gateway que acessa o ClusterIP

selector: app=api-tc

- Service roteia tráfego para pods com label `app=api-tc`
- Se houver 3 replicas, Service faz **load balancing** entre elas

port: 80, targetPort: 8080

- **port:** Porta exposta pelo Service (outros pods acessam via porta 80)
- **targetPort:** Porta que o container escuta (8080)

Fluxo de requisição:

Ingress (porta 80)



Service api-service-tc (porta 80)



Pod (porta 8080)

DNS interno do Kubernetes:

```
bash
```

```
# De qualquer pod no mesmo namespace
```

```
curl http://api-service-tc/api/products
```

```
# Resolve para: 10.240.0.X:80 (ClusterIP do Service)
```

04-api-hpa.yaml

```
yaml
```

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-hpa-tc
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-deployment-tc
  minReplicas: 1
  maxReplicas: 9
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
```

O que é: Escalonamento automático horizontal baseado em métricas.

Como funciona?

1. HPA monitora a CPU dos pods a cada 15 segundos
2. Se CPU média > 60%, HPA aumenta replicas

3. Se CPU média < 60%, HPA diminui replicas (após 5 min)

4. Nunca sai do intervalo [1, 9]

minReplicas: 1, maxReplicas: 9

- Sempre mantém pelo menos 1 pod rodando
- Pode escalar até 9 pods se necessário
- Node tem 2 CPUs (2000m):
 - Cada pod request: 100m
 - 9 pods = 900m = 45% de 1 node
 - 1 node suporta todos os 9 pods tranquilamente

averageUtilization: 60

- HPA calcula média de CPU de **todos os pods**
- Se média > 60%, adiciona pods
- Se média < 60%, remove pods (após cooldown)

Fórmula de escalonamento:

$\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} \times (\text{currentCPU} / \text{targetCPU}))$

Exemplo:

- 1 pod rodando
- CPU atual: 80%
- Target: 60%
- $\text{desiredReplicas} = \text{ceil}(1 \times (80/60)) = \text{ceil}(1.33) = 2$ pods

Cenário de carga:

10:00 - 1 pod @ 40% CPU → OK
10:05 - 1 pod @ 75% CPU → HPA cria 2º pod
10:08 - 2 pods @ 50% CPU → OK
10:15 - 2 pods @ 80% CPU → HPA cria 3º pod
10:20 - 3 pods @ 45% CPU → OK
10:30 - 3 pods @ 20% CPU → HPA remove 1 pod (após 5min)

Por que CPU e não memória?

- APIs .NET geralmente são **CPU-bound** (processamento)
- Memória é mais estável (não flutua tanto)

- Pode adicionar memória depois:

```
yaml

metrics:
- type: Resource
  resource:
    name: cpu
    target:
      averageUtilization: 60
- type: Resource
  resource:
    name: memory
    target:
      averageUtilization: 80
```

■ 05-api-ingress.yaml

```
yaml

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-ingress
  annotations:
    appgw.ingress.kubernetes.io/health-probe-path: "/"
spec:
  ingressClassName: azure-application-gateway
  rules:
  - host: api.internal.techchallenge
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: api-service-tc
            port:
              number: 80
```

O que é: Define regras de roteamento para o Application Gateway (AGIC).

ingressClassName: azure-application-gateway

- Define que o AGIC é responsável por processar este Ingress

- Sem isso, o Ingress é ignorado pelo AGIC
- Outras opções: `nginx`, `traefik` (se instalados)

annotations: health-probe-path: "/"

- Configura o **health check** do Application Gateway
- Gateway faz `GET http://10.0.1.11/` a cada 30s
- Se responder `200 OK`, backend é considerado Healthy
- Se responder `5xx` ou timeout, backend é Unhealthy

Onde configurar o health endpoint no .NET:

```
csharp

// Program.cs
app.MapHealthChecks("/");

// Ou endpoint específico
app.MapHealthChecks("/health");
// Então annotation seria: health-probe-path: "/health"
```

host: api.internal.techchallenge

- **DNS interno**, não público
- Usado pelo APIM para acessar o Application Gateway
- APIM resolve `api.internal.techchallenge` → IP privado do App Gateway (10.0.2.5)

Por que não usar IP diretamente no APIM?

- IPs privados podem mudar se o gateway for recriado
- Hostname é estável (configurado no Ingress)

path: /, pathType: Prefix

- **path: /** = Todas as rotas (catch-all)
- **pathType: Prefix** = Qualquer URL que comece com `/`

Exemplos de match:

-  `/api/products` → Match (começa com /)
-  `/api/orders/123` → Match
-  `/health` → Match

-   → Match

Se fosse path: /api:

-   /api/products → Match
-   /health → Não match

backend.service: api-service-tc:80

- Roteia tráfego para o **Service** (não diretamente para pods)
- Service faz load balancing entre os pods
- Application Gateway → Service → Pods

Fluxo Completo de Requisição

1. Cliente faz request:

`https://apim-tchungrgy-gateway-latest.azure-api.net/api/products`

2. APIM (subnet 10.0.3.X) resolve política e encaminha:

`http://api.internal.techchallenge/api/products`

3. DNS resolve api.internal.techchallenge → 10.0.2.5 (Application Gateway)

4. Application Gateway (10.0.2.5) verifica Ingress rules:

- Host: api.internal.techchallenge ✓
- Path: /api/products (match com /) ✓
- Backend: api-service-tc:80

5. App Gateway encaminha para Service ClusterIP (10.240.0.X:80)

6. Service faz load balancing e escolhe um pod:

- Pod 1: 10.0.1.11:8080
- Pod 2: 10.0.1.12:8080
- Escolhe Pod 1

7. Pod 1 processa a requisição:

- Container rodando .NET 8
- Acessa SQL (via connection string)
- Retorna JSON

8. Resposta volta pelo caminho inverso:

Pod → Service → App Gateway → APIM → Cliente

Recursos Criados por Estes Manifestos

Recurso	Nome	Tipo	Descrição
ConfigMap	api-configmap-tc	Config	Variáveis de ambiente
Deployment	api-deployment-tc	Workload	Gerencia pods
Pod	api-deployment-tc-xxx	Workload	Container rodando
Service	api-service-tc	Network	ClusterIP interno
HPA	api-hpa-tc	Autoscaling	Escalonamento auto
Ingress	api-ingress	Network	Regras de roteamento

Comandos Úteis

bash

Ver todos os recursos

kubectl get all -l **app**=api-tc

Ver ConfigMap

kubectl describe configmap api-configmap-tc

Ver logs do pod

kubectl logs -l **app**=api-tc --tail=**100** -f

Ver eventos do deployment

kubectl describe deployment api-deployment-tc

Forçar novo deploy (sem mudar código)

kubectl rollout restart deployment api-deployment-tc

Ver status do HPA

kubectl get hpa api-hpa-tc

Testar Service internamente (de outro pod)

kubectl run **test** --image=**curlimages/curl** -it --rm -- **curl** http://api-service-tc/health

Próximo passo: Documentar o repositório `infra-gateway` (Terraform do API Management).